

# Optimizing Fantasy Football Snake Draft

Massachusetts Institute of Technology

15.C57: Optimization Methods

Alexandre Jacquillat

Owen Bao, Justin Che, Justine Murdock

December 6, 2025

# 1 Problem Statement

Fantasy football has become one of the most popular strategy-based games in the United States, with more than 30 million players annually. One of the most important and challenging aspects of the game is the draft, where participants (managers) must assemble a competitive roster to maximize season-long performance (and avoid last-place penalties usually determined by their league). In snake drafts, the pick order reverses each round, meaning that player availability changes dynamically based on other managers’ decisions, preferences, and biases. Unlike in salary-cap formats, snake drafts require anticipating which players will still be available at future turns while balancing positional needs, bye weeks, and player injury risk.

Beyond its popularity, fantasy football provides an analytically meaningful test for optimization because the drafting process reflects key elements of real-world decision problems. Managers must make sequential choices with season-long consequences, operate under positional and roster constraints, anticipate competitor actions, and navigate substantial uncertainty in player outcomes. These characteristics make the fantasy football snake draft a rich environment for evaluating prescriptive analytics techniques, with insights that generalize to broader implementation and planning problems.

Our project formulates this decision process as an optimization problem that maximizes expected fantasy points subject to roster composition and availability constraints. We hope the resulting optimization framework will provide a data-driven approach to drafting that can outperform heuristic or expert-based strategies and yield more balanced and higher-scoring rosters.

## 2 Data Overview

We used publicly available FantasyPros 2025 preseason projection data, which include projected fantasy points, positional rankings, average draft position (ADP) for modeling draft availability by round, and player attributes such as position, team, and bye week.

Each position had its own datafile to be export, so in order to build a unified player dataset, we imported each positional CSV file (ex. QB, RB, WR, and TE) and combined them into a single table by stacking all rows together while retaining every column from each source file. Since each positional dataset had its own set of fields as different positions have different statistics associated with them, the merged table naturally contained many position-specific columns. However, when we combined the files, we found that two important attributes that we would look to use in our future modeling, injury risk and projected upside, did not carry over cleanly from the CSV imports due to being visualized as 1-D graphs of likelihood (a ranking of 1 to 5). To ensure these features were usable in our optimization model, we reconstructed and validated them after the merge, matching players across sources and filling in missing risk and upside values where necessary. This allowed us to preserve the full richness of the positional data while ensuring all players had complete and reliable attributes for decision-making.

## 3 Methods

### 3.1 Overview

Our modeling framework progresses from simple deterministic baselines to increasingly sophisticated optimization based approaches that incorporate risk and uncertainty. We began with three non-optimization baselines designed to establish intuitive drafting heuristics. Our first model is modeled after the ESPN Autodrafter, which drafts exclusively by Average Draft Position (ADP) while enforcing positional balance through rule-based roster logic, yielding a more realistic draft board without solving an optimization problem. The next baseline model ranks players purely by projected fantasy points, serving as a naïve “best available player” benchmark, while still enforcing positional balance. Our last baseline further enhances true “draft-like” conditions by incorporating ADP-adjusted availability, by modifying player rankings to reflect the likelihood that each player is attainable at a given pick. Additionally, this model also penalizing the objective with an ADP-based “reach penalty” if the optimizer tries to pick too far below the pick number. This model remains heuristic but introduces early notions of draft feasibility.

Once the appropriate baselines had been established, we shifted towards optimization. In the first true optimization model, we formulate the draft as a deterministic mixed-integer program that maximizes projected points subject to roster, positional, and pick-based constraints. Our next optimization model extends this optimization structure by integrating risk-aware features such as injury risk and projected upside directly into the objective, creating a deterministic but risk-adjusted optimization that allocates value more intelligently than our original optimization model. Finally, we introduced explicit uncertainty through scenario-based robust optimization. Due to computational complexity, we could not model all possible scenarios, but we simulated multiple possible performance outcomes and construct a draft that performs well across all of them, prioritizing downside protection under adverse scenarios. Together, these six models illustrate our systematic escalation from simple heuristics to fully robust decision-making under uncertainty.

### 3.2 Draft Structure Constraints

In addition to the roster requirements described later, all of our baseline and optimization models assume a common set of structural rules governing how players may be selected in a snake draft. These constraints ensure that each pick corresponds to a unique draft slot and that no player may be selected more than once across the league.

**One Player Per Pick.** At every pick number  $t$  in the draft, exactly one player is selected by the manager whose turn it is. Thus, for our own pick set  $\mathcal{T}$ , each pick must be assigned to exactly one available player.

**Players Cannot Be Drafted Twice.** Once a player is selected at any pick in the draft, they are no longer available for subsequent selections. All models therefore ensure that a player may be chosen at most once across the full draft horizon.

**Drafting Only at Assigned Picks.** In snake-draft formats, managers may only select players at their predetermined pick numbers (e.g., picks  $t \in \mathcal{T}$  for our team). Our models restrict all decision variables accordingly, preventing selections at picks not belonging to our draft order.

These structural constraints define the legal space of draft outcomes and apply uniformly across all baseline heuristics and optimization formulations. The roster constraints in the next section then govern which of these legal draft outcomes are positionally feasible.

### 3.3 Roster Constraints

All of our baseline and optimization models enforce a common set of roster constraints that ensure constructed teams are feasible under standard fantasy football rules. These constraints govern minimum positional requirements, maximum roster capacity, FLEX eligibility, and future feasibility when drafting sequentially.

**Minimum Positional Requirements.** Each drafted team must satisfy the following minimum roster requirements:

$$\text{min slots} = \{QB : 1, RB : 2, WR : 2, TE : 1, FLEX : 1, DST : 1, K : 1\}.$$

The FLEX position may be filled by any player in  $FLEX \text{ positions} = \{RB, WR, TE\}$ . Thus, at least one of the RB/WR/TE players must be eligible to fill the FLEX slot.

**Maximum Positional Limits.** No model is allowed to exceed the following positional caps:

$$\text{max slots} = \{QB : 2, RB : 5, WR : 5, TE : 2, K : 1, DST : 2\}.$$

These constraints prevent overdrafting a position (e.g., selecting excessive running backs or multiple kickers).

**Instantaneous Feasibility When Drafting.** Because players are selected sequentially, drafting a player at pick  $t$  must not make it impossible to satisfy minimum roster requirements with the remaining available picks. Let  $\text{current\_count}_p(t)$  denote the number of players currently drafted at position  $p$  and let  $R(t)$  be the number of picks remaining.

For a candidate player  $i$  with position  $p = \text{pos}(i)$ , we construct a hypothetical updated roster and compute whether the remaining minimums can be met:

$$\text{feasible\_pick}(p, \text{current\_count}, \text{min\_slots}, R(t) - 1) = 1.$$

Concretely, after tentatively adding player  $i$ , we verify:

1. All non-FLEX positions can still reach their minimums:

$$\text{current\_count}_q(t) + \mathbf{1}\{q = p\} + (R(t) - 1) \geq \text{min\_slots}_q, \quad \forall q \in \{QB, RB, WR, TE, DST, K\}.$$

2. The FLEX requirement remains satisfiable:

$$\sum_{q \in \{RB, WR, TE\}} (\text{current\_count}_q(t) + \mathbf{1}\{q = p\}) + (R(t) - 1) \geq 1.$$

This ensures that the team never drafts into a dead-end roster that cannot meet league requirements.

**Unified Application Across Models.** These positional constraints and feasibility rules are enforced across all our models. In every model, a candidate player is draftable only if both the maximum positional limit is respected and the future feasibility condition holds.

### 3.4 Baseline Models

We develop three heuristic baseline models that serve as practical reference points prior to introducing our optimization-based formulations. Each baseline selects players sequentially, applying the roster feasibility constraints described in the previous section. The baselines differ in the scoring logic used when choosing between eligible players.

**Baseline 0: ESPN-Style Autodraft.** This model emulates ESPN’s autodraft behavior by selecting players strictly in order of ESPN ADP (using ADP\_AVG when ESPN data is missing). At each of our picks  $t$ , we scan the ADP-sorted list and select the highest-ranked available player who satisfies roster feasibility. Formally,

$$i^{(t)} = \arg \min_{i \in A(t)} \text{ADP}_i^{\text{ESPN}} \quad \text{s.t. } i \text{ is roster-feasible.}$$

If no feasible player appears in ADP order, we select the next-highest-ranked available player. Opponents are assumed to draft strictly in ADP order. This baseline reflects a realistic autodraft mechanism driven entirely by consensus rankings.

**Baseline 1: Myopic FPTS Maximizer.** In contrast to ADP-driven drafting, this baseline selects at each pick the feasible player with the highest projected fantasy points (FPTS):

$$i^{(t)} = \arg \max_{i \text{ feasible}} \text{FPTS}_i.$$

This rule represents a rational but myopic drafter who values projections above all else and does not account for reaching risk or draft dynamics. Opponents again select strictly by ADP order.

**Baseline 2: Greedy Lookahead with Reaching Penalty.** The third baseline incorporates draft dynamics by penalizing “reaches” relative to ADP. The functional form of the ADP-based reach penalty, including its round weighting and calibration, is described in the Appendix alongside the supporting code implementation.

For each candidate player  $i$  at pick  $t$ , we compute a net value score:

$$\text{NetValue}_{i,t} = \text{FPTS}_i - \text{Penalty}(i, t),$$

where the penalty grows with the number of picks a player is drafted ahead of their ADP and is weighted more heavily in earlier rounds. Unlike the previous heuristics, this baseline uses a small mixed-integer program to optimize remaining picks:

$$\begin{aligned} \max_x \quad & \sum_{i,p} (\text{FPTS}_i - \text{Penalty}(i, p)) x_{i,p} \\ \text{s.t.} \quad & \sum_i x_{i,p} = 1, & \forall p \in \text{remaining picks}, \\ & \sum_p x_{i,p} \leq 1, & \forall i, \\ & x_{i,p} = 0 \text{ if } i \text{ violates roster feasibility,} \\ & x_{i,p} \in \{0, 1\}. \end{aligned}$$

We then draft the player assigned to the current pick  $t$ . This model balances projection strength against reaching cost, producing a more draft-aware selection strategy.

Together, these three baselines span a progression from consensus ranking adherence (Baseline 0) to a purely projection-driven decision making (Baseline 1) to a draft-aware model balancing value and reach (Baseline 2). They provide benchmarks for evaluating the benefits of our optimization models introduced in the next section.

### 3.5 Deterministic Mixed-Integer Optimization

Our first optimization-based formulation constructs a complete draft plan across all of our picks by solving a deterministic mixed-integer program (MIP). Unlike the baselines in Section 3.4, which select players one pick at a time, this model jointly optimizes all draft decisions while assuming that opponents pick strictly in ADP order. The model assigns players to specific pick numbers, enforces the roster rules from Section 3.3, and incorporates an ADP-based availability constraint that prevents selecting players who would implausibly remain on the board.

**Objective.** The model maximizes projected fantasy points while penalizing early-round reaches:

$$\max_x \sum_{i \in \mathcal{P}} \sum_{t \in \mathcal{T}} (\text{FPTS}_i - \text{Penalty}(i, t)) x_{i,t}.$$

The penalty term downweights players drafted significantly before their ADP and is weighted more heavily in early rounds, reflecting the higher value of early picks.

**Decision Variables.** The model uses two sets of binary decision variables:

$$x_{i,t} = \begin{cases} 1 & \text{if player } i \text{ is drafted at pick } t, \\ 0 & \text{otherwise,} \end{cases} \quad y_i = \begin{cases} 1 & \text{if player } i \text{ appears on the final roster,} \\ 0 & \text{otherwise.} \end{cases}$$

**Model Constraints.** All roster-feasibility requirements from Section 3.4 (minimum-position requirements, positional caps, FLEX eligibility, and instantaneous feasibility) are reused directly through the roster indicators  $y_i$ . We therefore only introduce constraints that are unique to the optimization formulation.

**Pick-Level Assignment.** Each pick must select exactly one player, and no player may be assigned to more than one pick:

$$\begin{aligned}\sum_{i \in \mathcal{P}} x_{i,t} &= 1, & \forall t \in \mathcal{T}, \\ \sum_{t \in \mathcal{T}} x_{i,t} &\leq 1, & \forall i \in \mathcal{P}.\end{aligned}$$

**Roster Linking.** To enable roster constraints to operate on  $y_i$ , we link pick assignments to roster inclusion:

$$y_i \geq \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} x_{i,t}, \quad y_i \leq \sum_{t \in \mathcal{T}} x_{i,t}, \quad \forall i \in \mathcal{P}.$$

**ADP-Based Availability.** Since opponents are assumed to draft strictly in ADP order, a player with ADP rank  $\text{ADP}_i$  cannot realistically remain available beyond any pick  $t > \text{ADP}_i$ . We therefore impose:

$$x_{i,t} = 0 \quad \text{if } t > \text{ADP}_i + \text{rank\_slack}.$$

This constraint is unique to the optimization model and enforces realistic draft-board evolution.

This deterministic MIP produces a full draft plan that is globally optimal under projected points and reach penalties. By capturing the draft as a structured assignment problem with roster feasibility, it provides a principled benchmark against which more sophisticated risk-aware and uncertainty-aware models can be evaluated.

### 3.6 Risk-Aware Deterministic Mixed-Integer Optimization

Our second optimization-based formulation extends the deterministic MIP by incorporating player-level risk attributes and structural roster safeguards. As in the previous model, opponents are assumed to draft strictly in ADP order, and all roster feasibility constraints from Section 3.2 and 3.3 (minimum/maximum positional requirements, FLEX eligibility, and future feasibility) apply unchanged. This model adds three additional components: (i) early-round quarterback discipline, (ii) bye-week diversification across skill positions, and (iii) a risk-adjusted objective that rewards upside and penalizes bust probability.

**Objective.** We augment projected fantasy points with risk adjustments. Each player  $i$  has projected points  $\text{FPTS}_i$ , an upside measure  $\text{Upside}_i$ , and a bust-risk measure  $\text{Bust}_i$ , and we apply fixed weights  $\lambda_{\text{up}}$  and  $\lambda_{\text{bust}}$ :

$$\max_x \sum_{i \in \mathcal{P}} \sum_{t \in \mathcal{T}} (\text{FPTS}_i - \text{Penalty}(i, t) + \lambda_{\text{up}} \cdot \text{Upside}_i - \lambda_{\text{bust}} \cdot \text{Bust}_i) x_{i,t}.$$

This creates a risk-aware optimizer that favors high-variance, high-upside players while avoiding extreme downside profiles. The construction of the upside and bust metrics, as well as the rationale behind the weighting parameters  $\lambda_{\text{up}}$  and  $\lambda_{\text{bust}}$ , is documented in the Appendix with full code for reproducibility.

**Decision Variables.** As in the prior model, we use pick-level assignment variables  $x_{i,t}$  and roster indicators  $y_i$ :

$$x_{i,t} \in \{0, 1\}, \quad y_i \in \{0, 1\}.$$

**Model Constraints.** Below, we restate only the constraints that are new to this formulation. All assignment constraints, uniqueness constraints, roster-linking constraints, and ADP-based availability restrictions appear identically in as the previous MIO and are not repeated.

**Early-Round Quarterback Discipline (New).** To prevent overcommitting to the quarterback position early in the draft, we restrict the model to selecting at most one quarterback within the first four picks:

$$\sum_{\substack{i \in \mathcal{P} \\ \text{pos}(i)=QB}} \sum_{t \in \mathcal{T}_{\text{top4}}} x_{i,t} \leq 1.$$

Here,  $\mathcal{T}_{\text{top4}}$  denotes our first four draft selections. This soft structural preference encourages more flexible roster construction in early rounds.

**Bye-Week Diversification (New).** To avoid lineup collapses during bye weeks, we restrict skill positions so that no two players at RB, WR, or TE share the same bye week:

$$\sum_{\substack{i \in \mathcal{P} \\ \text{pos}(i)=RB \\ \text{Bye}(i)=b}} y_i \leq 1, \quad \sum_{\substack{i \in \mathcal{P} \\ \text{pos}(i)=WR \\ \text{Bye}(i)=b}} y_i \leq 1, \quad \sum_{\substack{i \in \mathcal{P} \\ \text{pos}(i)=TE \\ \text{Bye}(i)=b}} y_i \leq 1, \quad \forall b.$$

These constraints explicitly hedge against correlated availability shocks and promote robustness in weekly lineup output.

**Roster Feasibility.** All constraints from Section 3.2 and 3.3 continue to apply directly through  $y_i$ , including minimum-slot requirements, positional caps, FLEX eligibility, and instantaneous feasibility across remaining picks. No changes are made to this logic in this MIO.

This model therefore produces a risk-aware optimal draft plan that preserves the structural logic of the previous model while incorporating upside, downside, early-round strategy rules, and bye-week diversification. This framework balances projected value with long-term risk exposure and improves roster stability over the baseline deterministic approach.

### 3.7 Scenario-Based Robust Optimization Model

Our final model incorporates explicit uncertainty by simulating full league draft scenarios and optimizing our draft strategy within each simulated environment. Instead of assuming deterministic opponent behavior, we generate multiple realizations of opponent picks and solve a separate optimization model for each scenario. The resulting framework produces a draft strategy that is robust to a wide range of possible draft outcomes.

**Objective Function.** As in our deterministic model, each scenario  $s \in \mathcal{S}$  solves a mixed-integer program that maximizes an adjusted player value score:

$$\max_x \sum_{i,s} \left( \text{FPTS}_i - \text{Penalty}(i,t) + \lambda_{\text{up}} \cdot \text{Up}_i - \lambda_{\text{bust}} \cdot \text{Bust}_i \right) x_{i,s}.$$

The reach penalty, upside reward, and bust penalty follow the formulations introduced in the previous model. Details of the penalty construction can be found in the accompanying code (see Appendix). Here, the objective is identical for all scenarios; what differs is the set of players who are available at each pick.

**Scenario Generation (New).** To reflect realistic opponent behavior, we simulate full 12-team drafts prior to optimization. Each scenario returns a mapping

$$\mathcal{A}^s(t) \subseteq \mathcal{P},$$

representing the set of players available at pick number  $t$  in scenario  $s$ . Opponent picks are generated probabilistically using round-dependent probability weights: early rounds heavily favor players near their ADP, while later rounds introduce more randomness. This reflects the common pattern of tight ADP adherence early and wider variation late in drafts.

We generate 30 independent scenarios ( $|\mathcal{S}| = 30$ ), which was sufficient to observe stable averages and variability in downstream results. Readers interested in the full implementation, including sampling weights, scenario logic, and reproducibility, may refer to the scenario-generation code in the Appendix.

**Decision Variables.** For each scenario, we optimize over the same binary variables used throughout our models:

$$x_{i,s} \in \{0, 1\}, \quad y_i = \mathbf{1}\left(\sum_s x_{i,s} = 1\right),$$

where  $x_{i,s}$  indicates whether we draft player  $i$  in our draft slot  $s$ , and  $y_i$  encodes whether player  $i$  appears anywhere on the final roster. These variables obey the assignment, uniqueness, and slot-linking relationships defined earlier.

**Scenario-Based Availability Constraints (New).** The key new structural feature of this model is a scenario-specific feasibility constraint. If scenario  $s$  predicts that player  $i$  has already been drafted before our pick number  $t$ , we enforce:

$$x_{i,s} = 0 \quad \text{whenever} \quad i \notin \mathcal{A}^s(t).$$

This replaces the deterministic ADP-based availability rule used in the earlier models. Each scenario therefore defines a distinct feasible region, depending on how opponents behave in that simulated draft.

**Roster and Structural Constraints.** All roster composition requirements, including minimum/maximum positional limits, FLEX eligibility, bye-week restrictions, and total roster size, are applied unchanged. Assignment and uniqueness constraints also remain identical to earlier optimization models. Therefore, differences in optimized outcomes arise purely from stochastic availability, not from modifications to roster structure.

**Scenario Evaluation and Representative Team.** After solving one mixed-integer program for each scenario, we collect the projected fantasy points  $\text{FPTS}^s$  for the resulting roster. We report summary statistics across all feasible scenarios (mean, variance, best and worst cases) to quantify sensitivity to draft randomness. To present a single team, we select the roster whose projected points are closest to the empirical average:

$$s = \arg \min_{s \in \mathcal{S}} \left| \text{FPTS}^s - \frac{1}{|\mathcal{S}|} \sum_{s' \in \mathcal{S}} \text{FPTS}^{s'} \right|.$$

This representative roster provides an interpretable summary of the robust solution and reflects the central tendency across several realistic draft realizations.

## 4 Key Findings and Discussion

These tables compare the performance of each drafting model at positions 3, 7, and 12 by reporting (1) projected preseason fantasy points, (2) actual points scored through Week 13 of the 2025-26 season, and (3) each roster’s resulting rank. Projected points reflect how strong a team was expected to be at the start of the season, while current points show how those projections translated into real performance. Ranking the models by actual scoring highlights which drafting strategies, heuristic, greedy with penalties, or optimization-based, produce the most reliable and highest-scoring teams once the season unfolds.

Table 1: Draft Slot 3: Projected vs. Current Fantasy Points

| Model          | Projected Points | Current Points | Rank |
|----------------|------------------|----------------|------|
| ESPN           | 1884.5           | 1274.8         | 4    |
| Myopic         | 1782.2           | 1070.9         | 6    |
| Greedy         | 1862.2           | 1602.4         | 1    |
| MIO            | 1876.7           | 1144.2         | 5    |
| Risk-Aware MIO | 1851.0           | 1334.7         | 3    |
| Robust         | 1913.8           | 1582.3         | 2    |



Table 2: Draft Slot 7: Projected vs. Current Fantasy Points

| Model          | Projected Points | Current Points | Rank |
|----------------|------------------|----------------|------|
| ESPN           | 1909.4           | 1482.9         | 3    |
| Myopic         | 1761.4           | 1064.3         | 6    |
| Greedy         | 1865.2           | 1330.5         | 5    |
| MIO            | 1857.8           | 1509.8         | 2    |
| Risk-Aware MIO | 1890.3           | 1608.9         | 1    |
| Robust         | 1905.6           | 1473.6         | 4    |

Table 3: Draft Slot 12: Projected vs. Current Fantasy Points

| Model          | Projected Points | Current Points | Rank |
|----------------|------------------|----------------|------|
| ESPN           | 1881.3           | 1372.2         | 3    |
| Myopic         | 1757.5           | 1018.1         | 6    |
| Greedy         | 1941.7           | 1317.8         | 4    |
| MIO            | 1879.7           | 1279.1         | 5    |
| Risk-Aware MIO | 1948.4           | 1404.7         | 2    |
| Robust         | 1959.8           | 1437.1         | 1    |

Across the three draft positions we analyzed (3, 7, and 12), clear performance patterns emerge that highlight the strengths and weaknesses of the six drafting models. While preseason projected totals for most models fall within a relatively narrow range, the realized in-season fantasy point totals (through Week 13 of the 2025–26 NFL season) vary substantially. This gap underscores the limitations of treating projections as deterministic estimates and highlights the value of incorporating uncertainty, risk, and structural constraints into drafting strategy.

## Key Findings Across Draft Positions

**Draft Position 3.** The Robust model entered with the highest projected points (1913.8) but finished slightly behind the Greedy Look-ahead model in realized performance (1582.3 vs. 1602.4). Meanwhile, the Myopic model, despite reasonable projected totals, finished last by a wide margin (1070.9), illustrating how projection-only strategies are fragile to injuries and underperformance. Models incorporating draft dynamics or uncertainty proved substantially more resilient.

**Draft Position 7.** This draft position exhibited the clearest separation between deterministic and uncertainty-aware optimization models. The risk-aware mixed integer optimization delivered the strongest realized outcome (1608.9), outperforming both heuristic models and the Robust approach. Middle draft positions often experience unpredictable positional runs, and the risk-aware model’s explicit balancing of upside and downside appears particularly effective in navigating this volatility. The Myopic model again performed worst, reinforcing its vulnerability to projection error.

**Draft Position 12.** At the turn, position 12, planning ahead becomes essential because picks occur in pairs, and failing to anticipate positional scarcity can lead to roster imbalance. Here, both optimization-based models with uncertainty (Risk-Aware Optimization Model and Robust) again delivered top realized scores (1404.7 and 1437.1). The Myopic model struggled once again, demonstrating that long-horizon planning is especially critical from boundary positions.

## Cross-Model Insights

Several overarching insights emerge across all draft positions:

**Risk-aware Deterministic Mixed Integer optimization excelled.** When positional runs were harder to predict (e.g., pick 7), risk-aware optimization performed the strongest, affirming the importance of representing variance, injury risk, and asymmetric player outcomes.

**Robust optimization was a close second model.** It produced similar results compared to the Risk-aware model. Incorporating opponent uncertainty and downside protection leads to more stable outcomes across diverse draft environments.

**Heuristic models struggled under real-world conditions.** The Myopic model finished last at all draft slots, highlighting the danger of taking projections at face value. The ESPN and Greedy models performed reasonably but lacked resilience to uncertainty.

**Projection-based drafting alone is insufficient.** Volatility in player performance, injuries, and changing team contexts introduce substantial noise. Models that explicitly incorporate uncertainty or structural risk far outperform those relying purely on preseason estimates.

## Interpretation and Implications

The combined results indicate that the most successful drafting strategies balance projection strength with uncertainty management and long-term structural planning. Incorporating risk-aware scoring (Risk-Aware Mixed-Integer Optimization) or opponent-behavior uncertainty (Robust) consistently improved in-season performance. These findings illustrate that in any sequential allocation problem, models that hedge against volatility and incorporate future opportunity costs systematically outperform those that rely on point estimates or static rankings.

## 5 Impact

While our project is motivated by the desire to draft better fantasy football rosters (and hopefully help all of us avoid last-place punishments next year), its broader value lies in showing how optimization and uncertainty modeling can meaningfully improve decision-making in settings that mirror real-world resource allocation problems. Fantasy football is an analytically meaningful test because drafting combines sequential choices, opportunity costs, positional constraints, and highly uncertain performance outcomes, features that closely resemble challenges faced in operations, the broader sports sector, and strategic planning. The draft also forces managers to anticipate competitor behavior, making it a natural environment for exploring prescriptive analytics.

The modeling framework we develop, progressing from heuristics to deterministic optimization to scenario-based robust decision-making, illustrates how incorporating uncertainty, risk profiles, and structural constraints leads to rosters that perform more consistently than purely projection-driven methods. Although our immediate application is recreational, the underlying techniques generalize to sports analytics contexts such as professional drafting, roster construction, or salary-cap optimization. More broadly, the tools used here, including mixed-integer programming and scenario-based robustness, extend to domains like workforce planning, and capital allocation, where decision quality improves when uncertainty is modeled explicitly rather than ignored. Ultimately, this project highlights how analytical optimization can transform both fantasy sports strategy and real-world decision-making and ideally help us win our leagues along the way.

## 6 Future Steps

If we were given another week, there would be several extensions that would meaningfully strengthen both the predictive foundation and the prescriptive performance of our drafting framework. A first direction involves replacing fixed projected fantasy points with a full predictive modeling pipeline capable of generating player-level outcome distributions. Rather than relying on imported upside and bust metrics, we could estimate empirical variance, injury probabilities, and positional correlation structures directly from historical data

using machine learning or other modeling approaches. For example, if a QB has a breakout, it is more likely for a WR on the same team to have a breakout as well. These distributions would enable more principled scenario generation for our stochastic and robust optimization models.

A second direction we could take is to extend our prescriptive component from a series of single-step optimizations into a fully multi-stage framework. The draft is inherently sequential, and the value of a pick is path-dependent. Methods such as extending the scenarios of our dynamic programming (albeit much more computationally expensive) could model the tradeoffs between immediate positional value and deferred opportunity across the remaining draft horizon. This would allow the optimization to account for future roster flexibility, positional scarcity, and evolving uncertainty as the draft unfolds.

A third direction we could take is opponent modeling. Our current baselines assume ADP-following opponents, but real drafters exhibit differing, non-stationary behavior. By estimating probabilistic pick distributions or clustering historical drafter types, we could incorporate opponent strategy into the prescriptive model, enabling robust or game-theoretic formulations that hedge against a range of draft environments. This would increase model realism and potentially shift optimal early-round strategies.

Finally, downstream evaluation represents a critical next step. Instead of evaluating models solely at draft time, we could simulate season-level outcomes using schedule-adjusted scoring distributions, replacement-level dynamics, and waiver-wire availability. Such a simulation would allow evaluation of expected wins, playoff probability, and value over replacement (VORP) under each drafting strategy. Integrating predictive modeling, multistage optimization, opponent behavior, and post-draft simulation would move our system toward an end-to-end predictive-prescriptive analytics pipeline for fantasy football roster construction.

Overall, while an additional week would not allow us to fully implement all of these extensions, the range of possible improvements highlights both the scalability of our current framework and the depth of opportunity that remains in prescriptive sports analytics. These directions illustrate not only how our draft model could evolve into a more comprehensive predictive-prescriptive system, but also the broader complexity in applying optimization, uncertainty modeling, and decision analysis to fantasy sports and real-world roster construction.

## Myopic Adaptive

```
In [21]: using CSV, DataFrames, Gurobi

# -----
# 1. Load & Clean
# -----

df = CSV.read("output.csv", DataFrame)

# Clean all string columns (safe)
for col in names(df)
    if eltype(df[!, col]) <: AbstractString
        df[!, col] = String.(df[!, col])
        df[!, col] = allowmissing(df[!, col])
        replace!(df[!, col], "-" => missing)
        replace!(df[!, col], "_" => missing)
        replace!(df[!, col], "" => missing)
    end
end

# Remove rows missing important model fields
df = filter(row -> !ismissing(row.FPTS) &&
              !ismissing(row.POSITION) &&
              !ismissing(row.ADP_AVG), df)

# Standard cleanup
df.POSITION = strip.(df.POSITION)
df.player_id = 1:nrow(df)

# -----
# 2. Create lookup dictionaries (KEEP df AS A DATAFRAME)
# -----

players = collect(df.player_id)           # player IDs
FPTS     = Dict{df.player_id, df.FPTS}    # id -> projection
pos      = Dict{df.player_id, df.POSITION} # id -> position
adp      = Dict{df.player_id, df.ADP_AVG}  # id -> ADP
bye_week = Dict{df.player_id, df.BYE_WEEK} # id -> bye week

# ★ NEW:
upside   = Dict{df.player_id, df.UPSIDE}  # id -> upside score
bust     = Dict{df.player_id, df.BUST}     # id -> bust score

# -----
# 3. Draft config
# -----

num_teams = 12
rounds     = 16
my_slot    = 7

T_max = num_teams * rounds
picks = 1:T_max

function get_snake_picks(my_slot::Int, num_teams::Int, roster_size::Int)
    picks = Int[]
    for r in 1:roster_size
        if isodd(r)
            push!(picks, (r-1)*num_teams + my_slot)
        else
            push!(picks, r*num_teams + 1 - my_slot)
        end
    end
    return picks
end

my_picks = get_snake_picks(my_slot, num_teams, rounds)

# -----
# 4. Roster constraints
# -----

min_slots = Dict{
    "QB" => 1,
    "RB" => 2,
    "WR" => 2,
    "TE" => 1,
    "FLEX" => 1,
    "DST" => 1,
    "K"  => 1
}

flex_positions = Set(["RB", "WR", "TE"])
```

```

max_slots = Dict(
    "QB" => 2,
    "RB" => 5,
    "WR" => 5,
    "TE" => 2,
    "K"  => 1,
    "DST" => 2
)

current_count = Dict(
    "QB" => 0,
    "RB" => 0,
    "WR" => 0,
    "TE" => 0,
    "K"  => 0,
    "DST" => 0
)

# -----
# 5. Feasibility function
# -----

function feasible_pick(p, current_count, min_slots, rounds_left)
    c = deepcopy(current_count)
    c[p] += 1

    remaining_required = 0

    for posname in keys(min_slots)
        if posname == "FLEX"
            needed = min_slots["FLEX"]
            have = sum(c[x] for x in ["RB", "WR", "TE"])
            if have < needed
                remaining_required += (needed - have)
            end
        else
            if c[posname] < min_slots[posname]
                remaining_required += (min_slots[posname] - c[posname])
            end
        end
    end

    return remaining_required <= rounds_left
end

# -----
# 6. Myopic Auto-Drafter 1 vs ADP opponents
# -----

sorted_players = sort(players, by = i -> adp[i])
taken = Set{Int}{}

my_team = DataFrame(
    pick=Int[], player_id=Int[], ADP=Float64[], PLAYER=String[],
    POSITION=String[], FPTS=Float64[], BYE_WEEK=Float64[]
)

for t in picks
    if t in my_picks

        available = [i for i in players if !(i in taken)]
        picks_left = length(my_picks[my_picks .>= t])

        eligible = [
            i for i in available
            if current_count[pos[i]] < max_slots[pos[i]] &&
               feasible_pick(pos[i], current_count, min_slots, picks_left - 1)
        ]

        if isempty(eligible)
            eligible = available
        end

        selected = argmax(i -> FPTS[i], eligible)

        push!(my_team, (
            t,
            selected,
            adp[selected],
            df.PLAYER[selected],
            pos[selected],
            FPTS[selected],
            bye_week[selected]
        ))

        current_count[pos[selected]] += 1
        push!(taken, selected)
    end
end

```

```

else
  # Opponents pick best available by ADP
  for i in sorted_players
    if !(i in taken)
      push!(taken, i)
      break
    end
  end
end
end
end

println("Myopic baseline team:")
println(my_team)
println("Total projected FPTS = ", sum(my_team.FPTS))
println("Position counts:", current_count)

```

Myopic baseline team:

16x7 DataFrame

| Row | pick  | player_id | ADP     | PLAYER              | POSITION | FPTS    | BYE_WEEK |
|-----|-------|-----------|---------|---------------------|----------|---------|----------|
|     | Int64 | Int64     | Float64 | String              | String   | Float64 | Float64  |
| 1   | 7     | 25        | 20.2    | Josh Allen          | QB       | 376.6   | 7.0      |
| 2   | 18    | 27        | 21.5    | Lamar Jackson       | QB       | 372.4   | 7.0      |
| 3   | 31    | 36        | 32.5    | Tyreek Hill         | WR       | 259.9   | 12.0     |
| 4   | 42    | 41        | 40.3    | Garrett Wilson      | WR       | 234.7   | 9.0      |
| 5   | 55    | 63        | 61.8    | Zay Flowers         | WR       | 224.4   | 7.0      |
| 6   | 66    | 81        | 74.5    | Jerry Jeudy         | WR       | 219.7   | 9.0      |
| 7   | 79    | 103       | 90.8    | Jakobi Meyers       | WR       | 197.9   | 8.0      |
| 8   | 90    | 183       | 107.2   | Brandon Aubrey      | K        | 140.1   | 10.0     |
| 9   | 103   | 112       | 112.8   | Austin Ekeler       | RB       | 123.4   | 12.0     |
| 10  | 114   | 158       | 158.0   | Denver Broncos      | DST      | 116.3   | 12.0     |
| 11  | 127   | 165       | 165.0   | Philadelphia Eagles | DST      | 116.3   | 9.0      |
| 12  | 138   | 143       | 157.7   | Jerome Ford         | RB       | 106.3   | 9.0      |
| 13  | 151   | 147       | 171.7   | Dylan Sampson       | RB       | 92.0    | 9.0      |
| 14  | 162   | 132       | 173.7   | Ray Davis           | RB       | 85.2    | 7.0      |
| 15  | 175   | 159       | 189.0   | Rico Dowdle         | RB       | 76.0    | 14.0     |
| 16  | 186   | 194       | 214.4   | Cade Otton          | TE       | 75.3    | 9.0      |

Total projected FPTS = 2816.5

Position counts:Dict{"QB" => 2, "TE" => 1, "DST" => 2, "RB" => 5, "WR" => 5, "K" => 1}

## ESPN Baseline

```

In [10]: println("\n===== ESPN-STYLE BASELINE (Baseline 0) =====")

# Reset roster counts
current_count = Dict{
  "QB" => 0,
  "RB" => 0,
  "WR" => 0,
  "TE" => 0,
  "K" => 0,
  "DST" => 0,
}

# ESPN fallback: if ESPN is missing, use ADP_AVG instead
df.unified_ESPN = [ismissing(df.ESPN[i]) ? df.ADP_AVG[i] : df.ESPN[i] for i in 1:nrow(df)]

# 🏈 ESPN-specific ADP ranking
# Replace :ESPN_ADP with the correct column name from your CSV
ESPN_rank = Dict{df.player_id .=> df.unified_ESPN}

sorted_players = sort(players, by = i -> ESPN_rank[i])

taken = Set{Int}()

espn_team = DataFrame(
  pick      = Int[],
  player_id = Int[],
  PLAYER    = String[],
  POSITION   = String[],
  FPTS      = Float64[],
  unified_ESPN = Float64[],
  BYE_WEEK  = Float64[]
)

for t in picks
  if t in my_picks
    available = [i for i in players if !(i in taken)]
    picks_left = length(my_picks[my_picks .>= t])

    selected = nothing

    # ESPN logic -> walk ESPN's ADP list top-down
    for i in sorted_players

```

```

        if i in taken
            continue
        end

        p = pos[i]

        if current_count[p] < max_slots[p] &&
            feasible_pick(p, current_count, min_slots, picks_left - 1)

            selected = i
            break
        end
    end

    if selected === nothing
        for i in sorted_players
            if !(i in taken)
                selected = i
                break
            end
        end
    end

    push!(espn_team, (
        t,
        selected,
        df.PLAYER[selected],
        pos[selected],
        FPTS[selected],
        ESPN_rank[selected],
        bye_week[selected]
    ))

    current_count[pos[selected]] += 1
    push!(taken, selected)

else
    for i in sorted_players
        if !(i in taken)
            push!(taken, i)
            break
        end
    end
end

println("\nESPN Autodraft Team:")
println(espn_team)
println("\nTotal projected FPTS = ", sum(espn_team.FPTS))
println("Position counts: ", current_count)

```

===== ESPN-STYLE BASELINE (Baseline 0) =====

ESPN Autodraft Team:

16x7 DataFrame

| Row | pick  | player_id | PLAYER              | POSITION | FPTS    | unified_ESPN | BYE_WEEK |
|-----|-------|-----------|---------------------|----------|---------|--------------|----------|
|     | Int64 | Int64     | String              | String   | Float64 | Float64      | Float64  |
| 1   | 7     | 13        | Christian McCaffrey | RB       | 233.0   | 7.0          | 14.0     |
| 2   | 18    | 18        | Josh Jacobs         | RB       | 229.5   | 18.0         | 5.0      |
| 3   | 31    | 36        | Tyreek Hill         | WR       | 259.9   | 31.0         | 12.0     |
| 4   | 42    | 64        | Patrick Mahomes II  | QB       | 328.1   | 42.0         | 10.0     |
| 5   | 55    | 55        | DJ Moore            | WR       | 231.1   | 55.0         | 5.0      |
| 6   | 66    | 45        | Jameson Williams    | WR       | 210.3   | 66.0         | 8.0      |
| 7   | 79    | 85        | Matthew Golden      | WR       | 191.6   | 79.0         | 5.0      |
| 8   | 90    | 124       | Cooper Kupp         | WR       | 193.0   | 91.0         | 8.0      |
| 9   | 103   | 183       | Brandon Aubrey      | K        | 140.1   | 107.0        | 10.0     |
| 10  | 114   | 77        | Jordan Mason        | RB       | 117.1   | 119.0        | 6.0      |
| 11  | 127   | 97        | Tank Bigsby         | RB       | 98.3    | 134.0        | 9.0      |
| 12  | 138   | 140       | Bhayshul Tuten      | RB       | 81.3    | 148.0        | 8.0      |
| 13  | 151   | 161       | Matthew Stafford    | QB       | 259.5   | 162.0        | 8.0      |
| 14  | 162   | 171       | Baltimore Ravens    | DST      | 114.8   | 171.0        | 7.0      |
| 15  | 175   | 181       | Buffalo Bills       | DST      | 115.9   | 181.0        | 7.0      |
| 16  | 186   | 175       | Brenton Strange     | TE       | 74.9    | 193.0        | 8.0      |

Total projected FPTS = 2878.4

Position counts: Dict{"QB" => 2, "TE" => 1, "DST" => 2, "RB" => 5, "WR" => 5, "K" => 1}

## Greedy Baseline

In [11]: using JuMP, Gurobi

```

println("\n===== GREEDY WITH REACHING PENALTY (Baseline 2) =====")

# -----
# Pick value calculation (Tuned Exponential Model)
# -----

```

```

function calculate_pick_value(pick_num::Int)
    return 400.0 * exp(-pick_num / 80.0)
end

# Build pick values dictionary
PICK_VALUES = Dict{Int, Float64}{}
for pick in 1:192
    PICK_VALUES[pick] = calculate_pick_value(pick)
end

println("✅ Using Tuned Exponential Model for Pick Values")

# -----
# Helper functions for penalty calculation
# -----

function acceptable_reach(pick_num::Int)
    round_num = ceil{Int}(pick_num / 12)

    if round_num == 1
        return 2.0
    elseif round_num == 2
        return 3.0
    elseif round_num == 3
        return 5.0
    elseif round_num == 4
        return 6.0
    elseif round_num <= 8
        return round_num + 4.0 # Rounds 5-8: 9, 10, 11, 12
    else
        return 15.0 + (round_num - 9) * 2 # Round 9: 15, Round 10: 17, etc.
    end
end

function penalty_weight(pick_num::Int)
    round_num = ceil{Int}(pick_num / 12)

    if round_num <= 3
        return 1.0 # Severe penalty rounds 1-3
    elseif round_num <= 6
        return 0.5 # Moderate penalty rounds 4-6
    elseif round_num <= 8
        return 0.2 # Light penalty rounds 7-8
    else
        return 0.02 # Nearly no penalty rounds 9+
    end
end

function calculate_penalty(player_id::Int, pick_num::Int)
    player_adp = adp[player_id]

    # How many picks early are we drafting them?
    picks_early = player_adp - pick_num

    # If picks_early <= 0, we're drafting at or after their ADP (value!), no penalty
    if picks_early <= 0
        return 0.0
    end

    # We're drafting early - check if it's within acceptable range
    acceptable = acceptable_reach(pick_num)

    if picks_early <= acceptable
        # Within acceptable reach, no penalty
        return 0.0
    else
        # Exceeds acceptable reach, apply penalty
        excess_reach = picks_early - acceptable
        λ = penalty_weight(pick_num)
        pick_val = get(PICK_VALUES, pick_num, 100.0)
        return λ * excess_reach * pick_val
    end
end

# -----
# Reset for new baseline
# -----

current_count = Dict{
    "QB" => 0,
    "RB" => 0,
    "WR" => 0,
    "TE" => 0,
    "K"  => 0,
    "DST" => 0,
}

```



```

sorted_players = sort(players, by = i -> adp[i])
taken = Set{Int}{}

greedy_team = DataFrame(
    pick      = Int[],
    player_id = Int[],
    PLAYER    = String[],
    POSITION   = String[],
    FPTS      = Float64[],
    ADP       = Float64[],
    penalty   = Float64[],
    net_value = Float64[],
    BYE_WEEK = Float64[]
)

# -----
# Main draft loop
# -----

for t in picks
    if t in my_picks
        println("\n--- Your pick: $t (Round $(ceil(Int, t/12))) ---")

        available = [i for i in players if !(i in taken)]
        remaining_picks = my_picks[my_picks .>= t]
        picks_left = length(remaining_picks)

        # Build optimization model for remaining picks
        model = Model(Gurobi.Optimizer)
        set_silent(model)

        # Decision variables: x[i,p] = 1 if player i selected at pick p
        @variable(model, x[i=available, p=remaining_picks], Bin)

        # Objective: maximize (FPTS - penalty)
        @objective(model, Max,
            sum((FPTS[i] - calculate_penalty(i, p)) * x[i,p]
                for i in available, p in remaining_picks)
        )

        # Constraint 1: Pick exactly one player at each of my remaining picks
        for p in remaining_picks
            @constraint(model, sum(x[i,p] for i in available) == 1)
        end

        # Constraint 2: Each player drafted at most once
        for i in available
            @constraint(model, sum(x[i,p] for p in remaining_picks) <= 1)
        end

        # Constraint 3: Position minimums (considering what we already have)
        for (position, min_needed) in min_slots
            if position == "FLEX"
                # FLEX means total RB+WR+TE >= 2+2+1+1 = 6
                flex_already = sum(current_count[pos_name] for pos_name in ["RB", "WR", "TE"])
                flex_needed = 2 + 2 + 1 + 1 # RB_min + WR_min + TE_min + FLEX_min

                # Filter flex-eligible players BEFORE the sum
                flex_available = [i for i in available if pos[i] in ["RB", "WR", "TE"]]

                @constraint(model,
                    sum(x[i,p] for i in flex_available, p in remaining_picks)
                    >= flex_needed - flex_already
                )
            else
                already_have = current_count[position]
                still_need = max(0, min_needed - already_have)

                # Filter position-specific players BEFORE the sum
                pos_available = [i for i in available if pos[i] == position]

                @constraint(model,
                    sum(x[i,p] for i in pos_available, p in remaining_picks)
                    >= still_need
                )
            end
        end

        # Constraint 4: Position maximums
        for (position, max_allowed) in max_slots
            already_have = current_count[position]
            can_still_draft = max_allowed - already_have

            # Filter position-specific players BEFORE the sum
            pos_available = [i for i in available if pos[i] == position]

            @constraint(model,
                sum(x[i,p] for i in pos_available, p in remaining_picks)

```

```

        )
        )
    end

    # Solve the optimization problem
    optimize!(model)

    if termination_status(model) != MOI.OPTIMAL
        println("Warning: No optimal solution found at pick $t")
        # Fallback to myopic if optimization fails
        eligible = [
            i for i in available
            if current_count[pos[i]] < max_slots[pos[i]] &&
               feasible_pick(pos[i], current_count, min_slots, picks_left - 1)
        ]
        selected = argmax(i -> FPTS[i], eligible)
    else
        # Extract the player selected for current pick t
        selected = nothing
        for i in available
            if value(x[i,t]) > 0.5
                selected = i
                break
            end
        end

        if selected === nothing
            println("Warning: No player selected at pick $t, using fallback")
            eligible = [
                i for i in available
                if current_count[pos[i]] < max_slots[pos[i]] &&
                   feasible_pick(pos[i], current_count, min_slots, picks_left - 1)
            ]
            selected = argmax(i -> FPTS[i], eligible)
        end
    end

    # Calculate penalty for selected player
    pen = calculate_penalty(selected, t)
    net_val = FPTS[selected] - pen

    println("Selected: $(df.PLAYER[selected]) ($(pos[selected])) - FPTS: $(FPTS[selected]), ADP: $(adp[selected])")

    push!(greedy_team, (
        t,
        selected,
        df.PLAYER[selected],
        pos[selected],
        FPTS[selected],
        adp[selected],
        pen,
        net_val,
        bye_week[selected]
    ))

    current_count[pos[selected]] += 1
    push!(taken, selected)

else
    # Opponents pick best available by ADP
    for i in sorted_players
        if !(i in taken)
            push!(taken, i)
            break
        end
    end
end
end
end

println("\n\n===== GREEDY TEAM RESULTS =====")
println(greedy_team)
println("\nTotal projected FPTS = ", sum(greedy_team.FPTS))
println("Total penalty = ", sum(greedy_team.penalty))
println("Net value = ", sum(greedy_team.net_value))
println("\nPosition counts: ", current_count)

```

===== GREEDY WITH REACHING PENALTY (Baseline 2) =====

✓ Using Tuned Exponential Model for Pick Values

--- Your pick: 7 (Round 1) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Christian McCaffrey (RB) - FPTS: 233.0, ADP: 8.2, Penalty: 0.0, Net: 233.0

--- Your pick: 18 (Round 2) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Brock Bowers (TE) - FPTS: 148.1, ADP: 20.0, Penalty: 0.0, Net: 148.1

--- Your pick: 31 (Round 3) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Tyreek Hill (WR) - FPTS: 259.9, ADP: 32.5, Penalty: 0.0, Net: 259.9

--- Your pick: 42 (Round 4) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Kenneth Walker III (RB) - FPTS: 178.4, ADP: 41.0, Penalty: 0.0, Net: 178.4

--- Your pick: 55 (Round 5) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Baker Mayfield (QB) - FPTS: 318.5, ADP: 62.5, Penalty: 0.0, Net: 318.5

--- Your pick: 66 (Round 6) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Aaron Jones Sr. (RB) - FPTS: 158.6, ADP: 65.3, Penalty: 0.0, Net: 158.6

--- Your pick: 79 (Round 7) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Deebo Samuel Sr. (WR) - FPTS: 197.6, ADP: 84.0, Penalty: 0.0, Net: 197.6

--- Your pick: 90 (Round 8) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Cooper Kupp (WR) - FPTS: 193.0, ADP: 92.5, Penalty: 0.0, Net: 193.0

--- Your pick: 103 (Round 9) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Austin Ekeler (RB) - FPTS: 123.4, ADP: 112.8, Penalty: 0.0, Net: 123.4

--- Your pick: 114 (Round 10) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Josh Downs (WR) - FPTS: 172.4, ADP: 127.3, Penalty: 0.0, Net: 172.4

--- Your pick: 127 (Round 11) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Darnell Mooney (WR) - FPTS: 174.2, ADP: 139.5, Penalty: 0.0, Net: 174.2

--- Your pick: 138 (Round 12) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Jerome Ford (RB) - FPTS: 106.3, ADP: 157.7, Penalty: 0.0, Net: 106.3

--- Your pick: 151 (Round 13) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Philadelphia Eagles (DST) - FPTS: 116.3, ADP: 165.0, Penalty: 0.0, Net: 116.3

--- Your pick: 162 (Round 14) ---

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license - for non-commercial use only - expires 2026-08-20

Selected: Buffalo Bills (DST) - FPTS: 115.9, ADP: 181.0, Penalty: 0.0, Net: 115.9

--- Your pick: 175 (Round 15) ---

```
Set parameter Username
Set parameter LicenseID to value 2697041
Academic license - for non-commercial use only - expires 2026-08-20
Selected: Bryce Young (QB) - FPTS: 263.9, ADP: 187.7, Penalty: 0.0, Net: 263.9

--- Your pick: 186 (Round 16) ---
Set parameter Username
Set parameter LicenseID to value 2697041
Academic license - for non-commercial use only - expires 2026-08-20
Selected: Matt Gay (K) - FPTS: 137.2, ADP: 193.2, Penalty: 0.0, Net: 137.2
```

===== GREEDY TEAM RESULTS =====

16x9 DataFrame

| Row | pick  | player_id | PLAYER              | POSITION | FPTS    | ADP     | penalty | net_value | BYE_WEEK |
|-----|-------|-----------|---------------------|----------|---------|---------|---------|-----------|----------|
|     | Int64 | Int64     | String              | String   | Float64 | Float64 | Float64 | Float64   | Float64  |
| 1   | 7     | 13        | Christian McCaffrey | RB       | 233.0   | 8.2     | 0.0     | 233.0     | 14.0     |
| 2   | 18    | 20        | Brock Bowers        | TE       | 148.1   | 20.0    | 0.0     | 148.1     | 8.0      |
| 3   | 31    | 36        | Tyreek Hill         | WR       | 259.9   | 32.5    | 0.0     | 259.9     | 12.0     |
| 4   | 42    | 40        | Kenneth Walker III  | RB       | 178.4   | 41.0    | 0.0     | 178.4     | 8.0      |
| 5   | 55    | 68        | Baker Mayfield      | QB       | 318.5   | 62.5    | 0.0     | 318.5     | 9.0      |
| 6   | 66    | 66        | Aaron Jones Sr.     | RB       | 158.6   | 65.3    | 0.0     | 158.6     | 6.0      |
| 7   | 79    | 84        | Deebo Samuel Sr.    | WR       | 197.6   | 84.0    | 0.0     | 197.6     | 12.0     |
| 8   | 90    | 124       | Cooper Kupp         | WR       | 193.0   | 92.5    | 0.0     | 193.0     | 8.0      |
| 9   | 103   | 112       | Austin Ekeler       | RB       | 123.4   | 112.8   | 0.0     | 123.4     | 12.0     |
| 10  | 114   | 118       | Josh Downs          | WR       | 172.4   | 127.3   | 0.0     | 172.4     | 11.0     |
| 11  | 127   | 121       | Darnell Mooney      | WR       | 174.2   | 139.5   | 0.0     | 174.2     | 5.0      |
| 12  | 138   | 143       | Jerome Ford         | RB       | 106.3   | 157.7   | 0.0     | 106.3     | 9.0      |
| 13  | 151   | 165       | Philadelphia Eagles | DST      | 116.3   | 165.0   | 0.0     | 116.3     | 9.0      |
| 14  | 162   | 181       | Buffalo Bills       | DST      | 115.9   | 181.0   | 0.0     | 115.9     | 7.0      |
| 15  | 175   | 142       | Bryce Young         | QB       | 263.9   | 187.7   | 0.0     | 263.9     | 14.0     |
| 16  | 186   | 241       | Matt Gay            | K        | 137.2   | 193.2   | 0.0     | 137.2     | 12.0     |

Total projected FPTS = 2896.7  
Total penalty = 0.0  
Net value = 2896.7

Position counts: Dict("QB" => 2, "TE" => 1, "DST" => 2, "RB" => 5, "WR" => 5, "K" => 1)

Greedy Drafter with Reaching Penalty - How It Works

Core Concept

The model optimizes draft picks by maximizing projected fantasy points while penalizing "reaches" - drafting players significantly earlier than their Average Draft Position (ADP).

Penalty Formula

```
Penalty = λ × Excess Reach × Pick Value

where:
- Excess Reach = max(0, (ADP - Current Pick) - Acceptable Reach)
- λ = Penalty weight (varies by round)
- Pick Value = Opportunity cost of the pick (exponential decay)
```

Key Components

1. Acceptable Reach (by round)
- Round 1: 2 picks
  - Round 2: 3 picks
  - Round 3: 5 picks
  - Round 4: 6 picks
  - Rounds 5-8: 9-12 picks (gradually increasing)
  - Rounds 9+: 15+ picks (very forgiving for sleepers)
2. Penalty Weight (λ)
- Rounds 1-3: λ = 1.0 (severe)
  - Rounds 4-6: λ = 0.5 (moderate)
  - Rounds 7-8: λ = 0.2 (light)
  - Rounds 9+: λ = 0.02 (minimal)
3. Pick Value (Tuned Exponential)

```
Pick Value = 400 × e^(-pick/80)
```

- Pick 7: ~366 points
- Pick 55: ~201 points

- Pick 127: ~82 points

Early picks have high opportunity cost; late picks are cheap to "waste."

## Examples

### Example 1: Big Reach Early (Bad)

- Draft QB at pick 7 with ADP 62.5
- Excess reach:  $62.5 - 7 - 2 = 53.5$  picks
- Penalty:  $1.0 \times 53.5 \times 366 = \mathbf{19,581 \text{ points}}$
- Result: Heavily discouraged

### Example 2: Small Reach (OK)

- Draft RB at pick 31 with ADP 35.8
- Excess reach:  $35.8 - 31 - 5 = 0$  picks
- Penalty: **0 points**
- Result: Within acceptable range

### Example 3: Late Round Sleeper (Fine)

- Draft player at pick 127 with ADP 180
- Excess reach:  $180 - 127 - 19 = 34$  picks
- Penalty:  $0.02 \times 34 \times 82 = \mathbf{56 \text{ points}}$
- Result: Small penalty, sleeper hunting encouraged

## Optimization 1: Static Optimization

```
In [22]: #####
# GLOBAL OPTIMIZATION MODEL (CONSISTENT WITH ADP BOTS) #
#####

using JuMP, Gurobi, DataFrames

# =====
# 1. Generate Your Pick Numbers Dynamically (snake draft)
# =====

function generate_my_picks(draft_slot::Int; teams::Int=12, rounds::Int=16)
    picks = Int[]
    for r in 1:rounds
        if isodd(r)
            # Odd rounds: order 1 → teams
            push!(picks, (r - 1) * teams + draft_slot)
        else
            # Even rounds: order teams → 1 (snake)
            push!(picks, r * teams - draft_slot + 1)
        end
    end
    return picks
end

# =====
# 2. Pre-compute ADP ranks (1 = earliest, N = latest)
# This matches the world where everyone else drafts by ADP.
# =====

# players, adp, FPTS, pos, bye_week, min_slots, max_slots, df
# are assumed to be already defined earlier in the notebook.

player_order = sort(players, by = i -> adp[i]) # ascending ADP
adp_rank = Dict{Int,Int}() # player_id -> ADP rank

for (k, i) in enumerate(player_order)
    adp_rank[i] = k
end

# If you want a tiny bit of optimism (allow a guy to fall a few spots past rank),
# you can set rank_slack > 0 below. rank_slack = 0 is exactly consistent with ADP bots.

# =====
# 3. Global Optimization Model (Model 1)
# =====

function solve_global_opt_model(
    players,
    my_picks;
    FPTS = FPTS,
```

```

pos      = pos,
adp      = adp,
bye_week = bye_week,
min_slots = min_slots,
max_slots = max_slots,
rank_slack = 0 # how many picks past ADP rank a player might "fall"
)

slots = collect(1:length(my_picks)) # your roster slots 1..R
slot_to_pick = Dict{s => my_picks[s] for s in slots} # slot -> overall pick number

model = Model(Gurobi.Optimizer)
set_silent(model)

# Decision variables:
# x[i,s] = 1 if player i is drafted by us in slot s
@variable(model, x[players, slots], Bin)

# y[i] = 1 if player i is on our final roster at any slot
@variable(model, y[players], Bin)

# -----
# A. Slot and Uniqueness Constraints
# -----

# One player per slot
for s in slots
    @constraint(model, sum(x[i,s] for i in players) == 1)
end

# Each player at most once
for i in players
    @constraint(model, sum(x[i,s] for s in slots) <= 1)
end

# Link x and y: if a player appears in any slot, y[i] = 1
for i in players
    @constraint(model, y[i] >= (1/length(slots)) * sum(x[i,s] for s in slots))
    @constraint(model, y[i] <= sum(x[i,s] for s in slots))
end

# -----
# B. ADP-Based Availability (Principled, ADP-Bot-Consistent)
# -----
# In a pure ADP-bot world:
# - A player with ADP rank r_i cannot still be on the board
#   after pick t > r_i.
#
# We implement:
# if pick_num > adp_rank[i] + rank_slack => x[i,s] = 0

for i in players, s in slots
    pick_num = slot_to_pick[s]
    r_i = adp_rank[i]

    if pick_num > r_i + rank_slack
        @constraint(model, x[i,s] == 0)
    end
end

# -----
# C. Position Feasibility Constraints (Baseline 2 logic)
# -----

# Minimums (including FLEX)
for (position, min_needed) in min_slots
    if position == "FLEX"
        # FLEX = any RB/WR/TE
        @constraint(model,
            sum(y[i] for i in players if pos[i] in ["RB", "WR", "TE"]) >= min_needed
        )
    else
        @constraint(model,
            sum(y[i] for i in players if pos[i] == position) >= min_needed
        )
    end
end

# Maximums (skip FLEX max if present)
for (position, max_allowed) in max_slots
    if position == "FLEX"
        continue
    end
    @constraint(model,
        sum(y[i] for i in players if pos[i] == position) <= max_allowed
    )
end

```

```

    )
end

# Total roster size equals number of our picks (e.g., 16)
@constraint(model, sum(y[i] for i in players) == length(slots))

# -----
# D. Objective: Maximize Fantasy Points – Reach Penalty
# -----
@objective(model, Max,
    sum(
        (FPTS[i] - calculate_penalty(i, slot_to_pick[s])) * x[i,s]
        for i in players, s in slots
    )
)

optimize!(model)

# -----
# E. Extract Optimized Team
# -----

opt_1_team = DataFrame(
    pick      = Int[],
    player_id = Int[],
    PLAYER    = String[],
    POSITION   = String[],
    FPTS      = Float64[],
    ADP       = Float64[],
    penalty   = Float64[],
    net_value = Float64[],
    BYE_WEEK  = Float64[]
)

for s in slots
    pick_num = slot_to_pick[s]

    chosen = nothing
    for i in players
        if value(x[i,s]) > 0.5
            chosen = i
            break
        end
    end

    if chosen === nothing
        # This should not happen if the model is feasible & optimal,
        # but we add a fallback for safety.
        remaining = setdiff(players, opt_1_team.player_id)
        chosen = argmax(i -> FPTS[i], remaining)
    end

    pen = calculate_penalty(chosen, pick_num)
    net_val = FPTS[chosen] - pen

    push!(opt_1_team, (
        pick_num,
        chosen,
        df.PLAYER[chosen],
        pos[chosen],
        FPTS[chosen],
        adp[chosen],
        pen,
        net_val,
        bye_week[chosen]
    ))
end

sort!(opt_1_team, :pick)
return opt_1_team
end

#####
# 4. Example Usage – Choose Any Draft Slot
#####

draft_slot = 7 # <--- change this freely from 1 to 12

my_picks = generate_my_picks(draft_slot)
println("\nYour picks from slot $draft_slot: ", my_picks)

global_opt_team = solve_global_opt_model(players, my_picks; rank_slack=0)

println("\n===== GLOBAL OPTIMIZED TEAM (Slot $draft_slot) =====")
println(global_opt_team[:, [:pick, :PLAYER, :POSITION, :ADP, :FPTS, :penalty, :net_value]])

```

```
println("\nTotal FPTS    = ", sum(global_opt_team.FPTS))
println("\nTotal Penalty = ", sum(global_opt_team.penalty))
println("\nNet Value     = ", sum(global_opt_team.net_value))
#####
```

Your picks from slot 7: [7, 18, 31, 42, 55, 66, 79, 90, 103, 114, 127, 138, 151, 162, 175, 186]

Set parameter Username

Set parameter LicenseID to value 2697041

Academic license – for non-commercial use only – expires 2026-08-20

===== GLOBAL OPTIMIZED TEAM (Slot 7) =====

16x7 DataFrame

| Row | pick  | PLAYER              | POSITION | ADP     | FPTS    | penalty | net_value |
|-----|-------|---------------------|----------|---------|---------|---------|-----------|
|     | Int64 | String              | String   | Float64 | Float64 | Float64 | Float64   |
| 1   | 7     | Christian McCaffrey | RB       | 8.2     | 233.0   | 0.0     | 233.0     |
| 2   | 18    | Jonathan Taylor     | RB       | 19.8    | 230.3   | 0.0     | 230.3     |
| 3   | 31    | Tyreek Hill         | WR       | 32.5    | 259.9   | 0.0     | 259.9     |
| 4   | 42    | James Conner        | RB       | 47.3    | 189.9   | 0.0     | 189.9     |
| 5   | 55    | David Montgomery    | RB       | 60.5    | 167.1   | 0.0     | 167.1     |
| 6   | 66    | Tony Pollard        | RB       | 65.2    | 171.6   | 0.0     | 171.6     |
| 7   | 79    | Kyler Murray        | QB       | 86.2    | 316.0   | 0.0     | 316.0     |
| 8   | 90    | Cooper Kupp         | WR       | 92.5    | 193.0   | 0.0     | 193.0     |
| 9   | 103   | Michael Pittman Jr. | WR       | 112.3   | 190.3   | 0.0     | 190.3     |
| 10  | 114   | Josh Downs          | WR       | 127.3   | 172.4   | 0.0     | 172.4     |
| 11  | 127   | Darnell Mooney      | WR       | 139.5   | 174.2   | 0.0     | 174.2     |
| 12  | 138   | J.J. McCarthy       | QB       | 146.0   | 292.1   | 0.0     | 292.1     |
| 13  | 151   | Hunter Henry        | TE       | 164.7   | 79.3    | 0.0     | 79.3      |
| 14  | 162   | Houston Texans      | DST      | 178.0   | 115.0   | 0.0     | 115.0     |
| 15  | 175   | Buffalo Bills       | DST      | 181.0   | 115.9   | 0.0     | 115.9     |
| 16  | 186   | Matt Gay            | K        | 193.2   | 137.2   | 0.0     | 137.2     |

Total FPTS = 3037.2000000000003

Total Penalty = 0.0

Net Value = 3037.2000000000003

## Optimization 2: Bye Week Constraint Optimization

```
In [23]: function solve_global_opt_model2(
    players,
    my_picks;
    FPTS      = FPTS,
    pos       = pos,
    adp       = adp,
    bye_week  = bye_week,
    upside    = upside,
    bust      = bust,
    min_slots = min_slots,
    max_slots = max_slots,
    rank_slack = 0 # how many picks past ADP rank a player might "fall"
)

    slots = collect(1:length(my_picks)) # your roster slots 1..R
    slot_to_pick = Dict{s => my_picks[s] for s in slots} # slot -> overall pick number

    model = Model(Gurobi.Optimizer)
    set_silent(model)

    # Decision variables:
    # x[i,s] = 1 if player i is drafted by us in slot s
    @variable(model, x[players, slots], Bin)

    # y[i] = 1 if player i is on our final roster at any slot
    @variable(model, y[players], Bin)

    # -----
    # A. Slot and Uniqueness Constraints
    # -----

    # One player per slot
    for s in slots
        @constraint(model, sum(x[i,s] for i in players) == 1)
    end

    # Each player at most once
    for i in players
        @constraint(model, sum(x[i,s] for s in slots) <= 1)
    end

    # Link x and y: if a player appears in any slot, y[i] = 1
    for i in players
        @constraint(model, y[i] >= (1/length(slots)) * sum(x[i,s] for s in slots))
        @constraint(model, y[i] <= sum(x[i,s] for s in slots))
    end

    # -----
```



```

# QB Constraint: At most 1 QB in your first 4 picks
# -----

top4_slots = slots[1:5]

@constraint(model,
    sum(x[i,s] for i in players for s in top4_slots if pos[i] == "QB") <= 1
)

# -----
# B. ADP-Based Availability (Principled, ADP-Bot-Consistent)
# -----

for i in players, s in slots
    pick_num = slot_to_pick[s]
    r_i = adp_rank[i]

    if pick_num > r_i + rank_slack
        @constraint(model, x[i,s] == 0)
    end
end

# -----
# C. Position Feasibility Constraints (Baseline 2 logic)
# -----

# Minimums (including FLEX)
for (position, min_needed) in min_slots
    if position == "FLEX"
        # FLEX = any RB/WR/TE
        @constraint(model,
            sum(y[i] for i in players if pos[i] in ["RB", "WR", "TE"]) >= min_needed
        )
    else
        @constraint(model,
            sum(y[i] for i in players if pos[i] == position) >= min_needed
        )
    end
end

# Maximums (skip FLEX max if present)
for (position, max_allowed) in max_slots
    if position == "FLEX"
        continue
    end
    @constraint(model,
        sum(y[i] for i in players if pos[i] == position) <= max_allowed
    )
end

# Total roster size equals number of our picks (e.g., 16)
@constraint(model, sum(y[i] for i in players) == length(slots))

# -----
# D. Bye Week Constraints (NEW FOR MODEL 2)
# -----
# No 2 RBs, no 2 WRs, and no 2 TEs share the same bye week.
# Collect all bye weeks among players
bye_weeks = unique([bye_week[i] for i in players])

for b in bye_weeks
    @constraint(model,
        sum(y[i] for i in players if pos[i] == "RB" && bye_week[i] == b) <= 1
    )
    @constraint(model,
        sum(y[i] for i in players if pos[i] == "WR" && bye_week[i] == b) <= 1
    )
    @constraint(model,
        sum(y[i] for i in players if pos[i] == "TE" && bye_week[i] == b) <= 1
    )
end

# -----
# E. Objective: Maximize Fantasy Points – Reach Penalty
# -----

lambda_up = 10.0      # reward upside
lambda_bust = 20.0    # penalize bust risk

@objective(model, Max,
    sum(
        (FPTS[i] - calculate_penalty(i, slot_to_pick[s])
        + lambda_up * upside[i]
        - lambda_bust * bust[i]) * x[i,s]
    )
)

```

```

        for i in players, s in slots
    )
)

optimize!(model)

# -----
# F. Extract Optimized Team
# -----

opt_2_team = DataFrame(
    pick      = Int[],
    player_id = Int[],
    PLAYER    = String[],
    POSITION   = String[],
    FPTs      = Float64[],
    ADP       = Float64[],
    penalty   = Float64[],
    net_value  = Float64[],
    BYE_WEEK  = Float64[]
)

for s in slots
    pick_num = slot_to_pick[s]

    chosen = nothing
    for i in players
        if value(x[i,s]) > 0.5
            chosen = i
            break
        end
    end

    if chosen === nothing
        # This should not happen if the model is feasible & optimal,
        # but we add a fallback for safety.
        remaining = setdiff(players, opt_2_team.player_id)
        chosen = argmax(i -> FPTs[i], remaining)
    end

    pen = calculate_penalty(chosen, pick_num)
    net_val = FPTs[chosen] - pen

    push!(opt_2_team, (
        pick_num,
        chosen,
        df.PLAYER[chosen],
        pos[chosen],
        FPTs[chosen],
        adp[chosen],
        pen,
        net_val,
        bye_week[chosen]
    ))
end

sort!(opt_2_team, :pick)
return opt_2_team
end

draft_slot = 7
my_picks = generate_my_picks(draft_slot)

global_opt_team2 = solve_global_opt_model2(players, my_picks; rank_slack=0)

println("\n===== GLOBAL OPTIMIZED TEAM (Model 2, Slot $draft_slot) =====")
println(global_opt_team2[:, [:pick, :PLAYER, :POSITION, :BYE_WEEK, :ADP, :FPTS, :penalty, :net_value]])

println("\nTotal FPTs      = ", sum(global_opt_team2.FPTS))
println("Total Penalty = ", sum(global_opt_team2.penalty))
println("Net Value      = ", sum(global_opt_team2.net_value))

```

Set parameter Username  
 Set parameter LicenseID to value 2697041  
 Academic license - for non-commercial use only - expires 2026-08-20

===== GLOBAL OPTIMIZED TEAM (Model 2, Slot 7) =====

16x8 DataFrame

| Row | pick  | PLAYER              | POSITION | BYE_WEEK | ADP     | FPTS    | penalty | net_value |
|-----|-------|---------------------|----------|----------|---------|---------|---------|-----------|
|     | Int64 | String              | String   | Float64  | Float64 | Float64 | Float64 | Float64   |
| 1   | 7     | Christian McCaffrey | RB       | 14.0     | 8.2     | 233.0   | 0.0     | 233.0     |
| 2   | 18    | A.J. Brown          | WR       | 9.0      | 20.3    | 257.6   | 0.0     | 257.6     |
| 3   | 31    | Jalen Hurts         | QB       | 9.0      | 35.2    | 358.7   | 0.0     | 358.7     |
| 4   | 42    | Marvin Harrison Jr. | WR       | 8.0      | 41.8    | 231.4   | 0.0     | 231.4     |
| 5   | 55    | RJ Harvey           | RB       | 12.0     | 60.8    | 152.0   | 0.0     | 152.0     |
| 6   | 66    | George Pickens      | WR       | 10.0     | 65.7    | 209.1   | 0.0     | 209.1     |
| 7   | 79    | Ricky Pearsall      | WR       | 14.0     | 87.5    | 187.6   | 0.0     | 187.6     |
| 8   | 90    | Jordan Mason        | RB       | 6.0      | 101.5   | 117.1   | 0.0     | 117.1     |
| 9   | 103   | Zach Charbonnet     | RB       | 8.0      | 110.0   | 119.2   | 0.0     | 119.2     |
| 10  | 114   | Tank Bigsby         | RB       | 9.0      | 135.2   | 98.3    | 8.08108 | 90.2189   |
| 11  | 127   | Darnell Mooney      | WR       | 5.0      | 139.5   | 174.2   | 0.0     | 174.2     |
| 12  | 138   | J.J. McCarthy       | QB       | 6.0      | 146.0   | 292.1   | 0.0     | 292.1     |
| 13  | 151   | Hunter Henry        | TE       | 14.0     | 164.7   | 79.3    | 0.0     | 79.3      |
| 14  | 162   | Houston Texans      | DST      | 6.0      | 178.0   | 115.0   | 0.0     | 115.0     |
| 15  | 175   | Buffalo Bills       | DST      | 7.0      | 181.0   | 115.9   | 0.0     | 115.9     |
| 16  | 186   | Matt Gay            | K        | 12.0     | 193.2   | 137.2   | 0.0     | 137.2     |

Total FPTS = 2877.7  
 Total Penalty = 8.081084363800274  
 Net Value = 2869.6189156361993

In [24]: using JuMP, Gurobi, DataFrames, StatsBase, Statistics

```
# -----
# STOCHASTIC OPPONENT MODELING
# -----

function generate_opponent_weights(round_num::Int)
    """
    Returns probability weights for opponent picks.
    Earlier rounds: stick closer to ADP
    Later rounds: more randomness
    """
    if round_num <= 3
        return [0.50, 0.25, 0.15, 0.07, 0.03] # Top pick 50% likely
    elseif round_num <= 6
        return [0.40, 0.25, 0.18, 0.10, 0.07]
    elseif round_num <= 10
        return [0.30, 0.25, 0.20, 0.15, 0.10]
    else
        return [0.25, 0.25, 0.20, 0.15, 0.15] # Late: almost uniform
    end
end

function simulate_one_scenario(players, my_picks, all_picks)
    """
    Simulate one complete scenario of opponent drafting.
    Returns: Dict mapping pick_number => Set of players still available
    """

    # Track which players are taken
    taken = Set{Int}{}

    # For each pick in the draft
    availability = Dict{Int, Set{Int}}{}

    for pick_num in all_picks
        # Store who's available at THIS pick (before it's made)
        availability[pick_num] = setdiff(Set(players), taken)

        if pick_num in my_picks
            # This is YOUR pick - don't simulate, you'll decide later
            # Just mark availability and continue
            continue
        else
            # Opponent's pick - simulate stochastically
            available_now = collect(setdiff(Set(players), taken))

            if isempty(available_now)
                continue
            end

            # Sort by ADP
            sorted_available = sort(available_now, by = i -> adp[i])

            # Determine round for weighting
            round_num = ceil{Int, pick_num / 12}
            weights = generate_opponent_weights(round_num)
        end
    end
end
```

```

        # Pick from top-K with weights
        K = min(5, length(sorted_available))
        top_k = sorted_available[1:K]
        pick_weights = weights[1:K]

        # Sample one player
        selected = sample(top_k, Weights(pick_weights))
        push!(taken, selected)
    end
end

return availability
end

function generate_scenarios(players, my_picks, all_picks, n_scenarios::Int)
    """
    Generate n_scenarios of opponent drafting behavior.
    Returns: Vector of availability dictionaries
    """

    println("\n🔄 Generating $n_scenarios draft scenarios...")
    scenarios = []

    for s in 1:n_scenarios
        scenario = simulate_one_scenario(players, my_picks, all_picks)
        push!(scenarios, scenario)

        if s % 10 == 0
            print(".")
        end
    end
    println(" Done!")

    return scenarios
end

# -----
# MODIFIED GLOBAL MODEL (with scenario-based availability)
# -----

function solve_global_opt_with_scenario(
    players,
    my_picks,
    availability_dict; # NEW: availability per pick from scenario
    FPTS = FPTS,
    pos = pos,
    adp = adp,
    bye_week = bye_week,
    upside = upside,
    bust = bust,
    min_slots = min_slots,
    max_slots = max_slots
)

    slots = collect(1:length(my_picks))
    slot_to_pick = Dict{s => my_picks[s] for s in slots}

    model = Model(Gurobi.Optimizer)
    set_silent(model)

    # Decision variables
    @variable(model, x[players, slots], Bin)
    @variable(model, y[players], Bin)

    # -----
    # A. Slot and Uniqueness Constraints
    # -----

    for s in slots
        @constraint(model, sum(x[i,s] for i in players) == 1)
    end

    for i in players
        @constraint(model, sum(x[i,s] for s in slots) <= 1)
    end

    for i in players
        @constraint(model, y[i] >= (1/length(slots)) * sum(x[i,s] for s in slots))
        @constraint(model, y[i] <= sum(x[i,s] for s in slots))
    end

    # -----
    # A2. QB Constraint: At most 1 QB in first 5 picks
    # -----

    top5_slots = slots[1:min(5, length(slots))]
    @constraint(model,
        sum(x[i,s] for i in players for s in top5_slots if pos[i] == "QB") <= 1
    )

```

```

)

# -----
# B. SCENARIO-BASED AVAILABILITY (NEW!)
# -----
# Instead of deterministic ADP + slack, use scenario availability

for s in slots
    pick_num = slot_to_pick[s]
    available_at_pick = availability_dict[pick_num]

    # If player i is NOT available in this scenario, force x[i,s] = 0
    for i in players
        if !(i in available_at_pick)
            @constraint(model, x[i,s] == 0)
        end
    end
end

# -----
# C. Position Feasibility Constraints
# -----

for (position, min_needed) in min_slots
    if position == "FLEX"
        @constraint(model,
            sum(y[i] for i in players if pos[i] in ["RB", "WR", "TE"]) >= min_needed
        )
    else
        @constraint(model,
            sum(y[i] for i in players if pos[i] == position) >= min_needed
        )
    end
end

for (position, max_allowed) in max_slots
    if position == "FLEX"
        continue
    end
    @constraint(model,
        sum(y[i] for i in players if pos[i] == position) <= max_allowed
    )
end

@constraint(model, sum(y[i] for i in players) == length(slots))

# -----
# D. Bye Week Constraints
# -----

bye_weeks = unique([bye_week[i] for i in players])

for b in bye_weeks
    @constraint(model,
        sum(y[i] for i in players if pos[i] == "RB" && bye_week[i] == b) <= 1
    )
    @constraint(model,
        sum(y[i] for i in players if pos[i] == "WR" && bye_week[i] == b) <= 1
    )
    @constraint(model,
        sum(y[i] for i in players if pos[i] == "TE" && bye_week[i] == b) <= 1
    )
end

# -----
# E. Objective: Maximize Fantasy Points with penalties
# -----

lambda_up = 10.0
lambda_bust = 20.0

@objective(model, Max,
    sum(
        (FPTS[i] - calculate_penalty(i, slot_to_pick[s])
        + lambda_up * upside[i]
        - lambda_bust * bust[i]) * x[i,s]
        for i in players, s in slots
    )
)

optimize!(model)

# -----
# F. Extract Solution
# -----

if termination_status(model) != MOI.OPTIMAL
    return nothing, -Inf # Infeasible scenario
end

```

```

end

team = DataFrame(
    pick      = Int[],
    player_id = Int[],
    PLAYER     = String[],
    POSITION    = String[],
    FPTS       = Float64[],
    ADP        = Float64[],
    penalty    = Float64[],
    net_value  = Float64[],
    BYE_WEEK   = Float64[]
)

for s in slots
    pick_num = slot_to_pick[s]

    chosen = nothing
    for i in players
        if value(x[i,s]) > 0.5
            chosen = i
            break
        end
    end

    if chosen === nothing
        return nothing, -Inf
    end

    pen = calculate_penalty(chosen, pick_num)
    net_val = FPTS[chosen] - pen

    push!(team, (
        pick_num,
        chosen,
        df.PLAYER[chosen],
        pos[chosen],
        FPTS[chosen],
        adp[chosen],
        pen,
        net_val,
        bye_week[chosen]
    ))
end

sort!(team, :pick)
obj_val = objective_value(model)

return team, obj_val
end

# -----
# MAIN STOCHASTIC ROBUST OPTIMIZATION
# -----

function solve_stochastic_robust_model(
    players,
    my_picks,
    all_picks;
    n_scenarios = 30
)

println("\n" * "="^70)
println("STOCHASTIC ROBUST OPTIMIZATION WITH $n_scenarios SCENARIOS")
println("="^70)

# Step 1: Generate scenarios
scenarios = generate_scenarios(players, my_picks, all_picks, n_scenarios)

# Step 2: Solve for each scenario
println("\n\ Solving optimization for each scenario...")

scenario_teams = []
scenario_values = []
scenario_fpts = []

feasible_count = 0

for (sc_idx, scenario) in enumerate(scenarios)
    team, obj_val = solve_global_opt_with_scenario(
        players,
        my_picks,
        scenario
    )

    if team != nothing
        push!(scenario_teams, team)
        push!(scenario_values, obj_val)
    end
end

```

```

        push!(scenario_fpts, sum(team.FPTS))
        feasible_count += 1
    end

    if sc_idx % 5 == 0
        print(".")
    end
end
println(" Done!")

println("\n✅ Feasible scenarios: $feasible_count / $n_scenarios")

if feasible_count == 0
    println("❌ No feasible solutions found!")
    return nothing
end

# Step 3: Analyze results across scenarios
println("\n" * "="^70)
println("SCENARIO ANALYSIS")
println("="^70)

avg_fpts = mean(scenario_fpts)
std_fpts = std(scenario_fpts)
min_fpts = minimum(scenario_fpts)
max_fpts = maximum(scenario_fpts)

println("Expected FPTS:      $(round(avg_fpts, digits=2))")
println("Std Dev FPTS:        $(round(std_fpts, digits=2))")
println("Min FPTS:             $(round(min_fpts, digits=2))")
println("Max FPTS:             $(round(max_fpts, digits=2))")
println("Range:                $(round(max_fpts - min_fpts, digits=2))")

# Step 4: Select representative team
# Strategy: Choose team closest to expected value

expected_idx = argmin(abs.(scenario_fpts .- avg_fpts))
representative_team = scenario_teams[expected_idx]

println("\n🏆 Selected representative team (closest to expected value)")
println("Scenario #$expected_idx with FPTS = $(round(scenario_fpts[expected_idx], digits=2))")

# Step 5: Player frequency analysis
println("\n" * "="^70)
println("PLAYER SELECTION FREQUENCY ACROSS SCENARIOS")
println("="^70)

player_frequency = Dict{Int, Int}()
for team in scenario_teams
    for player_id in team.player_id
        player_frequency[player_id] = get(player_frequency, player_id, 0) + 1
    end
end

# Sort by frequency
freq_sorted = sort(collect(player_frequency), by = x -> x[2], rev = true)

println("\nTop players by selection frequency:")
for (i, (player_id, count)) in enumerate(freq_sorted[1:min(20, length(freq_sorted))])
    pct = round(100 * count / feasible_count, digits=1)
    println("  $(df.PLAYER[player_id]) ($(pos[player_id])): $count/$feasible_count ($pct%)")

    if i >= 20
        break
    end
end

# Return results
return Dict(
    "representative_team" => representative_team,
    "all_teams" => scenario_teams,
    "all_values" => scenario_values,
    "all_fpts" => scenario_fpts,
    "avg_fpts" => avg_fpts,
    "std_fpts" => std_fpts,
    "min_fpts" => min_fpts,
    "max_fpts" => max_fpts,
    "player_frequency" => player_frequency,
    "n_feasible" => feasible_count
)
end

# -----
# RUN THE STOCHASTIC ROBUST MODEL
# -----

println("\n\n" * "="^70)
println("RUNNING STOCHASTIC ROBUST GLOBAL OPTIMIZATION")

```

```

println("═^70)

draft_slot = 7
my_picks = generate_my_picks(draft_slot)
all_picks = collect(1:192) # Assuming 16 rounds × 12 teams

# Test with fewer scenarios first
n_scenarios_test = 10
println("\n✓ Test run with $n_scenarios_test scenarios...")
test_results = solve_stochastic_robust_model(
    players,
    my_picks,
    all_picks;
    n_scenarios = n_scenarios_test
)

if test_results != nothing
    println("\n✓ Test successful! Running full optimization...")

    # Full run with 30 scenarios
    n_scenarios_full = 30
    final_results = solve_stochastic_robust_model(
        players,
        my_picks,
        all_picks;
        n_scenarios = n_scenarios_full
    )

    if final_results != nothing
        println("\n\n" * "═^70)
        println("FINAL STOCHASTIC ROBUST TEAM")
        println("═^70)

        final_team = final_results["representative_team"]
        println(final_team[:, [:pick, :PLAYER, :POSITION, :BYE_WEEK, :ADP, :FPTS, :penalty, :net_value]])

        println("\n✓ Performance Statistics:")
        println("Expected FPTS:  ${(round(final_results["avg_fpts"], digits=2))}")
        println("Std Dev:      ${(round(final_results["std_fpts"], digits=2))}")
        println("Best Case:     ${(round(final_results["max_fpts"], digits=2))}")
        println("Worst Case:    ${(round(final_results["min_fpts"], digits=2))}")

        # Save for comparison
        global stochastic_robust_team = final_team
        global stochastic_results = final_results
    end
else
    println("\n✗ Test failed – check model constraints")
end
end

```



## RUNNING STOCHASTIC ROBUST GLOBAL OPTIMIZATION

🔧 Test run with 10 scenarios...

STOCHASTIC ROBUST OPTIMIZATION WITH 10 SCENARIOS

```
🎲 Generating 10 draft scenarios...
. Done!
```

🔧 Solving optimization for each scenario...

[illegible]

✓ Feasible scenarios: 10 / 10

## SCENARIO ANALYSIS

|                |         |
|----------------|---------|
| Expected FPTS: | 2941.88 |
| Std Dev FPTS:  | 30.05   |
| Min FPTS:      | 2908.5  |
| Max FPTS:      | 3004.7  |
| Range:         | 96.2    |

Selected representative team (closest to expected value)  
Scenario #7 with FPTs = 2943.8

### PLAYER SELECTION FREQUENCY ACROSS SCENARIOS

Top players by selection frequency:

|                           |                |
|---------------------------|----------------|
| Zach Charbonnet (RB):     | 10/10 (100.0%) |
| J.J. McCarthy (QB):       | 10/10 (100.0%) |
| Buffalo Bills (DST):      | 10/10 (100.0%) |
| Kaleb Johnson (RB):       | 8/10 (80.0%)   |
| Jalen Hurts (QB):         | 8/10 (80.0%)   |
| Drake London (WR):        | 8/10 (80.0%)   |
| Zach Ertz (TE):           | 7/10 (70.0%)   |
| Tank Bigsby (RB):         | 7/10 (70.0%)   |
| Davante Adams (WR):       | 6/10 (60.0%)   |
| Christian McCaffrey (RB): | 6/10 (60.0%)   |
| Denver Broncos (DST):     | 6/10 (60.0%)   |
| Ray Davis (RB):           | 6/10 (60.0%)   |
| Courtland Sutton (WR):    | 6/10 (60.0%)   |
| George Pickens (WR):      | 5/10 (50.0%)   |
| Brandon Aubrey (K):       | 5/10 (50.0%)   |
| DeVonta Smith (WR):       | 4/10 (40.0%)   |
| Ka'imi Fairbairn (K):     | 4/10 (40.0%)   |
| Houston Texans (DST):     | 4/10 (40.0%)   |
| Jordan Mason (RB):        | 3/10 (30.0%)   |
| Ricky Pearsall (WR):      | 3/10 (30.0%)   |

✅ Test successful! Running full optimization...

23/31

Set parameter LicenseID to value 2697041  
 Academic license – for non-commercial use only – expires 2026-08-20  
 Set parameter Username  
 Set parameter LicenseID to value 2697041  
 Academic license – for non-commercial use only – expires 2026-08-20  
 Set parameter Username  
 Set parameter LicenseID to value 2697041  
 Academic license – for non-commercial use only – expires 2026-08-20  
 Set parameter Username  
 Set parameter LicenseID to value 2697041  
 Academic license – for non-commercial use only – expires 2026-08-20  
 . Done!

✓ Feasible scenarios: 30 / 30

#### SCENARIO ANALYSIS

Expected FPTS: 2951.35  
 Std Dev FPTS: 22.41  
 Min FPTS: 2891.8  
 Max FPTS: 2998.7  
 Range: 106.9

📊 Selected representative team (closest to expected value)  
 Scenario #25 with FPTS = 2951.6

#### PLAYER SELECTION FREQUENCY ACROSS SCENARIOS

Top players by selection frequency:  
 J.J. McCarthy (QB): 30/30 (100.0%)  
 Buffalo Bills (DST): 30/30 (100.0%)  
 Zach Charbonnet (RB): 29/30 (96.7%)  
 Jalen Hurts (QB): 27/30 (90.0%)  
 Tank Bigsby (RB): 25/30 (83.3%)  
 Davante Adams (WR): 23/30 (76.7%)  
 Kaleb Johnson (RB): 23/30 (76.7%)  
 Zach Ertz (TE): 23/30 (76.7%)  
 Denver Broncos (DST): 20/30 (66.7%)  
 Drake London (WR): 20/30 (66.7%)  
 Jordan Mason (RB): 16/30 (53.3%)  
 Courtland Sutton (WR): 16/30 (53.3%)  
 Justin Jefferson (WR): 15/30 (50.0%)  
 George Pickens (WR): 15/30 (50.0%)  
 Ka'imi Fairbairn (K): 15/30 (50.0%)  
 Ray Davis (RB): 14/30 (46.7%)  
 DeVonta Smith (WR): 12/30 (40.0%)  
 Ricky Pearsall (WR): 11/30 (36.7%)  
 Houston Texans (DST): 10/30 (33.3%)  
 Brandon Aubrey (K): 9/30 (30.0%)

#### FINAL STOCHASTIC ROBUST TEAM

16×8 DataFrame

| Row | pick  | PLAYER           | POSITION | BYE_WEEK | ADP     | FPTS    | penalty | net_value |
|-----|-------|------------------|----------|----------|---------|---------|---------|-----------|
|     | Int64 | String           | String   | Float64  | Float64 | Float64 | Float64 | Float64   |
| 1   | 7     | CeeDee Lamb      | WR       | 10.0     | 5.0     | 306.1   | 0.0     | 306.1     |
| 2   | 18    | A.J. Brown       | WR       | 9.0      | 20.3    | 257.6   | 0.0     | 257.6     |
| 3   | 31    | Jalen Hurts      | QB       | 9.0      | 35.2    | 358.7   | 0.0     | 358.7     |
| 4   | 42    | Davante Adams    | WR       | 8.0      | 40.3    | 243.8   | 0.0     | 243.8     |
| 5   | 55    | Courtland Sutton | WR       | 12.0     | 52.8    | 222.1   | 0.0     | 222.1     |
| 6   | 66    | RJ Harvey        | RB       | 12.0     | 60.8    | 152.0   | 0.0     | 152.0     |
| 7   | 79    | Kaleb Johnson    | RB       | 5.0      | 81.2    | 146.1   | 0.0     | 146.1     |
| 8   | 90    | Matthew Golden   | WR       | 5.0      | 92.2    | 191.6   | 0.0     | 191.6     |
| 9   | 103   | Jordan Mason     | RB       | 6.0      | 101.5   | 117.1   | 0.0     | 117.1     |
| 10  | 114   | Zach Charbonnet  | RB       | 8.0      | 110.0   | 119.2   | 0.0     | 119.2     |
| 11  | 127   | Cameron Dicker   | K        | 12.0     | 133.5   | 131.3   | 0.0     | 131.3     |
| 12  | 138   | Tank Bigsby      | RB       | 9.0      | 135.2   | 98.3    | 0.0     | 98.3      |
| 13  | 151   | J.J. McCarthy    | QB       | 6.0      | 146.0   | 292.1   | 0.0     | 292.1     |
| 14  | 162   | Zach Ertz        | TE       | 12.0     | 154.2   | 84.7    | 0.0     | 84.7      |
| 15  | 175   | Buffalo Bills    | DST      | 7.0      | 181.0   | 115.9   | 0.0     | 115.9     |
| 16  | 186   | Houston Texans   | DST      | 6.0      | 178.0   | 115.0   | 0.0     | 115.0     |

📊 Performance Statistics:  
 Expected FPTS: 2951.35  
 Std Dev: 22.41  
 Best Case: 2998.7  
 Worst Case: 2891.8

```
Out[24]: Dict{String, Any} with 10 entries:
  "representative_team" => 16x9 DataFrame...
  "all_values"          => Any[3070.3, 3066.4, 3071.9, 3071.5, 3080.1, 3031.8, ...
  "avg_fpts"            => 2951.35
  "player_frequency"    => Dict{5=>1, 56=>4, 35=>23, 6=>6, 115=>1, 112=>8, 90=>...
  "n_feasible"          => 30
  "min_fpts"            => 2891.8
  "std_fpts"            => 22.4112
  "all_teams"           => Any[16x9 DataFrame...
  "max_fpts"            => 2998.7
  "all_fpts"            => Any[2960.3, 2926.4, 2931.9, 2931.5, 2940.1, 2891.8, ...
```

```
In [25]: #####
# STARTER EVALUATION FUNCTION (WORKS WITH ALL BASELINES)
#####
const STARTING_RULES = Dict{
  "QB" => 1,
  "RB" => 2,
  "WR" => 2,
  "TE" => 1,
  "K"  => 1,
  "DST" => 1,
  "FLEX" => 1
}
const FLEX_POOL = Set{String}(["RB", "WR", "TE"])

"""
Evaluate starters for a given drafted roster.
Returns: (starter_dataframe, total_fpts)
"""
function evaluate_starters(team::DataFrame; proj_col::Symbol = :FPTS)
  starters = DataFrame()

  # ===== NON-FLEX STARTERS =====
  for (position, count) in STARTING_RULES
    if position == "FLEX"
      continue
    end

    pos_pool = filter(row -> row.POSITION == position, team)
    if nrow(pos_pool) == 0
      continue
    end

    pos_pool_sorted = sort(pos_pool, proj_col, rev=true)
    # Take top players
    selected = pos_pool_sorted[1:min(count, nrow(pos_pool_sorted)), :]
    append!(starters, DataFrame(selected))
  end

  # ===== FLEX STARTER =====
  flex_pool = filter(
    row -> (row.POSITION in FLEX_POOL) && !(row.player_id in starters.player_id),
    team
  )
  if nrow(flex_pool) > 0
    best_flex = sort(flex_pool, proj_col, rev=true)[1:1, :]
    append!(starters, DataFrame(best_flex))
  end

  total_fpts = sum(starters[:, proj_col])
  return starters, total_fpts
end

#####
# RUN EVALUATION FOR ALL BASELINES
#####
# Evaluate starters for every model
espn_starters,   espn_points   = evaluate_starters(espn_team)
myopic_starters, myopic_points = evaluate_starters(my_team)
greedy_starters, greedy_points = evaluate_starters(greedy_team)
opt_1_starters,  opt_1_points  = evaluate_starters(global_opt_team)
opt_2_starters,  opt_2_points  = evaluate_starters(global_opt_team2)

println("\n" * "-"^60)
println("STARTER RESULTS")
println("-"^60)
println("ESPN (Baseline 0) Starter Points:      ", round(espn_points, digits=1))
println("Myopic (Baseline 1) Starter Points:      ", round(myopic_points, digits=1))
println("Greedy w/ Penalty (Baseline 2) Starter Points: ", round(greedy_points, digits=1))
println("Static Optimization (Model 1) Starter Points: ", round(opt_1_points, digits=1))
println("Adaptive Optimization (Model 2) Starter Points: ", round(opt_2_points, digits=1))

# -----
# Detailed comparison table (Now includes Adaptive Model 2)
# -----
comparison = DataFrame(
```

```

Model = [
    "ESPN (Baseline 0)",
    "Myopic (Baseline 1)",
    "Greedy w/ Penalty (Baseline 2)",
    "Static Optimization (Model 1)",
    "Adaptive Optimization (Model 2)"
],
Starter_Points = [
    espn_points,
    myopic_points,
    greedy_points,
    opt_1_points,
    opt_2_points
],
Total_Roster_Points = [
    sum(espn_team.FPTS),
    sum(my_team.FPTS),
    sum(greedy_team.FPTS),
    sum(global_opt_team.FPTS),
    sum(global_opt_team2.FPTS)
],
Bench_Depth = [
    sum(espn_team.FPTS)           - espn_points,
    sum(my_team.FPTS)            - myopic_points,
    sum(greedy_team.FPTS)        - greedy_points,
    sum(global_opt_team.FPTS)    - opt_1_points,
    sum(global_opt_team2.FPTS)   - opt_2_points
]
)

# Ranking by starter points
comparison.Starter_Rank = sortperm(comparison.Starter_Points, rev=true)

println("\n" * "="^60)
println("DETAILED COMPARISON")
println("="^60)
println(comparison)

# -----
# Improvements over ESPN baseline
# -----
espn_base = espn_points

println("\n" * "="^60)
println("IMPROVEMENTS OVER ESPN BASELINE")
println("="^60)
println("Myopic improvement:           +$(round(myopic_points - espn_base, digits=1)) points ($(round((myopic_poin
println("Greedy improvement:           +$(round(greedy_points - espn_base, digits=1)) points ($(round((greedy_poi
println("Static Optimization improvement: +$(round(opt_1_points - espn_base, digits=1)) points ($(round((opt_1_poin
println("Adaptive Optimization improvement: +$(round(opt_2_points - espn_base, digits=1)) points ($(round((opt_2_po

if greedy_points > myopic_points
    println("\n✅ Greedy w/ Penalty BEATS Myopic by $(round(greedy_points - myopic_points, digits=1)) points!")
elseif myopic_points > greedy_points
    println("\n⚠️ Myopic beats Greedy by $(round(myopic_points - greedy_points, digits=1)) points")
else
    println("\n🤝 Greedy and Myopic tied")
end

println("="^60)

```

```

=====
WARNING: redefinition of constant Main.STARTING_RULES. This may fail, cause incorrect answers, or produce other errors.
WARNING: redefinition of constant Main.FLEX_POOL. This may fail, cause incorrect answers, or produce other errors.

```

## STARTER RESULTS

```
=====
ESPN (Baseline 0) Starter Points:      1822.8
Myopic (Baseline 1) Starter Points:    1657.0
Greedy w/ Penalty (Baseline 2) Starter Points: 1782.0
Static Optimization (Model 1) Starter Points: 1754.9
Adaptive Optimization (Model 2) Starter Points: 1774.2
=====
```

## DETAILED COMPARISON

5x5 DataFrame

| Row | Model String                    | Starter_Points<br>Float64 | Total_Roster_Points<br>Float64 | Bench_Depth<br>Float64 | Starter_Rank<br>Int64 |
|-----|---------------------------------|---------------------------|--------------------------------|------------------------|-----------------------|
| 1   | ESPN (Baseline 0)               | 1822.8                    | 2878.4                         | 1055.6                 | 1                     |
| 2   | Myopic (Baseline 1)             | 1657.0                    | 2816.5                         | 1159.5                 | 3                     |
| 3   | Greedy w/ Penalty (Baseline 2)  | 1782.0                    | 2896.7                         | 1114.7                 | 5                     |
| 4   | Static Optimization (Model 1)   | 1754.9                    | 3037.2                         | 1282.3                 | 4                     |
| 5   | Adaptive Optimization (Model 2) | 1774.2                    | 2877.7                         | 1103.5                 | 2                     |

## IMPROVEMENTS OVER ESPN BASELINE

```
=====
Myopic improvement:      +-165.8 points (-9.1%)
Greedy improvement:      +-40.8 points (-2.2%)
Static Optimization improvement: +-67.9 points (-3.7%)
Adaptive Optimization improvement: +-48.6 points (-2.7%)
=====
```

✅ Greedy w/ Penalty BEATS Myopic by 125.0 points!

In [26]: using JuMP, Gurobi

```
#####
Evaluate team accounting for bye weeks using optimization for fill-ins.
For each week, solves an assignment problem to maximize lineup points.
#####
function evaluate_with_byes(team::DataFrame)

    # Step 1: Identify starters (using the simple evaluation)
    starters, _ = evaluate_starters(team)

    # Step 2: Identify bench (everyone not in starters)
    bench = filter(row -> !(row.player_id in starters.player_id), team)

    total_points_across_season = 0.0
    weekly_breakdown = DataFrame{
        Week = Int[],
        Points = Float64[],
        Byes = String[]
    }

    # Step 3: Iterate through all 18 weeks
    for week in 1:18

        # Identify who's on bye this week
        starters_on_bye = filter(row -> row.BYE_WEEK == Float64(week), starters)
        starters_active = filter(row -> row.BYE_WEEK != Float64(week), starters)

        # Available bench players (not on bye)
        available_bench = filter(row -> row.BYE_WEEK != Float64(week), bench)

        # If no one is on bye, just use starters
        if nrow(starters_on_bye) == 0
            week_points = sum(starters.FPTS) / 17.0
            total_points_across_season += week_points
            push!(weekly_breakdown, (week, week_points, "None"))
            continue
        end

        # Build optimization model for this week
        model = Model{Gurobi.Optimizer}
        set_silent(model)

        # All candidates = active starters + available bench
        all_candidates = vcat(starters_active, available_bench)

        # Define slots we need to fill
        slots = ["QB", "RB1", "RB2", "WR1", "WR2", "TE", "FLEX", "K", "DST"]

        # Decision variables: x[i, s] = 1 if player i fills slot s
        @variable(model, x[1:nrow(all_candidates), slots], Bin)

        # Objective: maximize total weekly points
        @objective(model, Max,
            sum(x[i, s] * (all_candidates.FPTS[i] / 17.0)
                for i in 1:nrow(all_candidates), s in slots)
        )
    end
end
```

```

)

# Constraint 1: Each slot filled by exactly one player (or 0 if no one available)
for s in slots
  @constraint(model, sum(x[i, s] for i in 1:nrow(all_candidates)) <= 1)
end

# Constraint 2: Each player assigned to at most one slot
for i in 1:nrow(all_candidates)
  @constraint(model, sum(x[i, s] for s in slots) <= 1)
end

# Constraint 3: Position eligibility
for i in 1:nrow(all_candidates)
  player_pos = all_candidates.POSITION[i]

  # Only QBs can fill QB slot
  if player_pos != "QB"
    @constraint(model, x[i, "QB"] == 0)
  end

  # Only Ks can fill K slot
  if player_pos != "K"
    @constraint(model, x[i, "K"] == 0)
  end

  # Only DSTs can fill DST slot
  if player_pos != "DST"
    @constraint(model, x[i, "DST"] == 0)
  end

  # Only TEs can fill TE slot (but can also go in FLEX)
  if player_pos != "TE"
    @constraint(model, x[i, "TE"] == 0)
  end

  # RBs can only fill RB slots or FLEX
  if player_pos == "RB"
    @constraint(model, x[i, "QB"] + x[i, "WR1"] + x[i, "WR2"] + x[i, "TE"] + x[i, "K"] + x[i, "DST"] == 0)
  end

  # WRs can only fill WR slots or FLEX
  if player_pos == "WR"
    @constraint(model, x[i, "QB"] + x[i, "RB1"] + x[i, "RB2"] + x[i, "TE"] + x[i, "K"] + x[i, "DST"] == 0)
  end

  # TEs can fill TE or FLEX only
  if player_pos == "TE"
    @constraint(model, x[i, "QB"] + x[i, "RB1"] + x[i, "RB2"] + x[i, "WR1"] + x[i, "WR2"] + x[i, "K"] + x[i, "DST"] == 0)
  end
end

# Solve
optimize!(model)

if termination_status(model) == MOI.OPTIMAL
  week_points = objective_value(model)
else
  # Fallback: just count active starters
  week_points = sum(starters_active.FPTS) / 17.0
  println("Warning: No optimal solution for week $week")
end

total_points_across_season += week_points

bye_players = nrow(starters_on_bye) > 0 ? join(starters_on_bye.PLAYER, ", ") : "None"
push!(weekly_breakdown, (week, week_points, bye_players))
end

return total_points_across_season, weekly_breakdown
end

#####
# Run bye week evaluation for all teams (including Opt 1 & Opt 2)
#####

println("\n" * "="^70)
println("BYE WEEK ADJUSTED EVALUATION")
println("="^70)

# Evaluate teams using your bye-week simulation
espn_bye_points, espn_weekly = evaluate_with_byes(espn_team)
myopic_bye_points, myopic_weekly = evaluate_with_byes(my_team)
greedy_bye_points, greedy_weekly = evaluate_with_byes(greedy_team)
opt1_bye_points, opt1_weekly = evaluate_with_byes(global_opt_team)
opt2_bye_points, opt2_weekly = evaluate_with_byes(global_opt_team2)

println("\nESPN (Baseline 0) - Bye-Adjusted Points: $(round(espn_bye_points, digits=1))")

```

```

println("Myopic (Baseline 1) - Bye-Adjusted Points:      $(round(myopic_bye_points, digits=1))")
println("Greedy (Baseline 2) - Bye-Adjusted Points:      $(round(greedy_bye_points, digits=1))")
println("Static Optimization (Model 1) - Bye-Adjusted:    $(round(opt1_bye_points, digits=1))")
println("Adaptive Optimization (Model 2) - Bye-Adjusted:  $(round(opt2_bye_points, digits=1))")

# Build comparison table
bye_comparison = DataFrame(
  Model = [
    "ESPN",
    "Myopic",
    "Greedy w/ Penalty",
    "Static Optimization",
    "Adaptive Optimization"
  ],
  Bye_Adjusted_Points = [
    espn_bye_points,
    myopic_bye_points,
    greedy_bye_points,
    opt1_bye_points,
    opt2_bye_points
  ],
  Simple_Starter_Points = [
    espn_points,
    myopic_points,
    greedy_points,
    opt_1_points,
    opt_2_points      # Must compute earlier via evaluate_starters(opt_2_team)
  ],
  Depth_Quality = [
    espn_bye_points - espn_points,
    myopic_bye_points - myopic_points,
    greedy_bye_points - greedy_points,
    opt1_bye_points - opt_1_points,
    opt2_bye_points - opt_2_points
  ]
)

# Rank by bye-adjusted points (higher = better)
bye_comparison.Rank = sortperm(bye_comparison.Bye_Adjusted_Points, rev=true)

println("\n" * "="^70)
println("FINAL COMPARISON WITH BYE WEEKS")
println("="^70)
println(bye_comparison)

println("\n" * "="^70)
println("ANALYSIS")
println("="^70)
println("Depth Quality shows the difference between bye-adjusted and simple evaluation.")
println("Negative values mean the team struggled during bye weeks (poor depth).")
println("Less negative values mean better depth and bye week management.")
println("="^70)

```



|                                                 |        |
|-------------------------------------------------|--------|
| ESPN (Baseline 0) - Bye-Adjusted Points:        | 1909.4 |
| Myopic (Baseline 1) - Bye-Adjusted Points:      | 1761.4 |
| Greedy (Baseline 2) - Bye-Adjusted Points:      | 1865.2 |
| Static Optimization (Model 1) - Bye-Adjusted:   | 1857.8 |
| Adaptive Optimization (Model 2) - Bye-Adjusted: | 1890.3 |

30/31

5x5 DataFrame

| Row | Model                 | Bye_Adjusted_Points | Simple_Starter_Points | Depth_Quality | Rank  |
|-----|-----------------------|---------------------|-----------------------|---------------|-------|
|     | String                | Float64             | Float64               | Float64       | Int64 |
| 1   | ESPN                  | 1909.38             | 1822.8                | 86.5765       | 1     |
| 2   | Myopic                | 1761.38             | 1657.0                | 104.382       | 5     |
| 3   | Greedy w/ Penalty     | 1865.21             | 1782.0                | 83.2118       | 3     |
| 4   | Static Optimization   | 1857.8              | 1754.9                | 102.9         | 4     |
| 5   | Adaptive Optimization | 1890.34             | 1774.2                | 116.141       | 2     |

ANALYSIS

Depth Quality shows the difference between bye-adjusted and simple evaluation.  
Negative values mean the team struggled during bye weeks (poor depth).  
Less negative values mean better depth and bye week management.

In [ ]: